Statically Verifying Well-Definedness of Separately Compiled Attribute Grammars

FEDERICO BRUZZONE, Università degli Studi di Milano, Italy
WALTER CAZZOLA*, Università degli Studi di Milano, Italy
LUCA FAVALLI, Università degli Studi di Milano, Italy

The development of programming languages involves complex theoretical and practical challenges, particularly when addressing modularity and reusability through language extensions. While language workbenches aim to enable modular development under the constraints of the language extension problem, one critical constraint—separate compilation—is often relaxed due to its complexity. However, this relaxation undermines artifact reusability and integration with common dependency systems. A key difficulty under separate compilation arises from managing attribute grammars, as extensions may introduce new attributes that invalidate previously generated abstract syntax tree structures. Existing approaches, such as the use of dynamic maps in the `Neverlang` workbench, favor flexibility at the cost of compile-time correctness, leading to potential runtime errors due to undefined attributes. This work addresses this issue by introducing `nlgcheck`, a theoretically sound static analysis tool based on data-flow analysis for the `Neverlang` language workbench, which detects potential runtime errors—such as undefined attribute accesses—when separately compiled language components are composed, before the resulting language is ever run, thus preserving separate compilation while maintaining strong static correctness guarantees. Experimental evaluation using mutation testing on `Neverlang`-based projects demonstrates that `nlgcheck` effectively enhances robustness without sacrificing modularity or flexibility and with a level of performance that does not impede its adoption in daily development activities.

CCS Concepts: • **Theory of computation** → **Program analysis**; • **Software and its engineering** → *Compilers*.

Additional Key Words and Phrases: language composition, separate compilation, modular language design, sound compositional semantics, domain-specific languages, language frameworks, attribute grammars, static analysis

ACM Reference Format:
Federico Bruzzone, Walter Cazzola, and Luca Favalli. 2026. Statically Verifying Well-Definedness of Separately Compiled Attribute Grammars. *ACM Trans. Program. Lang. Syst.* 1, 1 (November 2026), 56 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Development of programming languages is a complex activity that requires mastery of both theoretical and practical concepts spanning aspects such as grammars, parsers, optimization techniques,

*Corresponding author.

Authors' Contact Information: Federico Bruzzone, federico.bruzzone@unimi.it, Università degli Studi di Milano, Computer Science Department, Milan, Italy, Italy; Walter Cazzola, cazzola@di.unimi.it, cazzola@di.unimi.it, Università degli Studi di Milano, Computer Science Department, Milan, Italy, Italy; Luca Favalli, luca.favalli@unimi.it, Università degli Studi di Milano, Computer Science Department, Milan, Italy, Italy.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1558-4593/2026/11-ART
https://doi.org/XXXXXXX.XXXXXXX

architectures, and software engineering. This is true for both *general-purpose languages* (GPL) and *domain-specific languages* (DSL) although with varying degrees of complexity and different challenges: where GPLs are usually larger and more complex projects overall, DSLs require deep knowledge of the application domain and often involve more interaction with domain experts that may or may not coincide with the end users [73].

Although each programming language is unique and has been designed for a specific purpose, it has been observed that many provide similar language constructs among them, due to recurring modeling patterns and abstractions [26, 72]. In this context, the challenge for language designers is to take advantage of such commonalities by reusing any existing software artifacts as much as possible. In this regard, the *language extension problem* [68] is a popular variant of the even more popular *expression problem* [94], modified to suit the needs and constraints of programming languages and their extension. Among the tools for language implementation, *language workbenches* [42], such as `Neverlang` [20, 25, 86], strive to enable purely modular development and composition techniques by adhering to the constraints of the language extension problem and by providing the means to perform language composition at several levels of granularity [10, 38].

Despite the efforts made to achieve this result, one among the constraints of the language extension problem is often (if not always) disregarded. In the authors' own words [68]: «the constraint of *separate compilation* usually impacts other non-functional properties (…). Consequently, it can be worthwhile to relax the separate compilation constraint in order to comply with other non-functional properties». This reflects a common tension: joint (*i.e.*, non-separate) compilation can enable whole-program optimizations and simplifies certain correctness guarantees, since the compiler has access to the implementation of every language component at once, at the cost of the modularity and independent shippability that separate compilation provides. While the increase in complexity related to separate compilation is undeniable, we believe that relaxing this constraint can severely hinder the actual reusability of language artifacts. For instance, without separate compilation, language extension is only possible when source code is available, whereas some among the most popular repositories for dependency management, such as `Maven` Central[1] and `NuGet`,[2] distribute libraries as compiled artifacts, thus limiting the possibility to reuse language libraries from the web.

Attribute grammars (AGs) [61] are a well-established formalism for specifying the semantics of programming languages. An AG extends a context-free grammar by attaching *attributes* to its symbols and *semantic equations* to its productions: each equation defines the value of an attribute in terms of other attributes occurring in the same production. Attributes whose values are computed from the children of a node, and thus flow upward in the tree, are called *synthesized*. Attributes whose values are computed from the parent or the siblings of a node, and thus flow downward or sideways, are called *inherited* (§2.1). Compilers use AGs to implement the analyses and translations performed on the *abstract syntax tree* (AST) of a program, such as name analysis, type checking, and code generation [1, 76]. Among the complexities brought by separate compilation, we believe the most pressing one to be the implementation of AGs. In most language workbenches, the compiler generates data structures for the AST nodes that may appear in the language under development.[3] Then, semantic actions [1] can be associated with each node to implement the semantics. However, to properly create the data structure for each AST node, it is necessary to foresee all the grammar attributes that may be inherited or synthesized by that node, as each

[1] https://central.sonatype.com
[2] https://www.nuget.org
[3] For simplicity, we describe this with respect to a single, unified AST representation; in practice, a compiler is typically organized into several phases, each with its own intermediate representation, but the argument that follows applies equally to each of them.

attribute will need a corresponding field within that data structure. It is apparent that when separate compilation is permitted, a language construct may be extended with more semantic actions at a later time, possibly adding more attributes, thus rendering the AST data structures obsolete.

Some language workbenches with separate compilation, such as `Neverlang`, simply ignore this problem. In `Neverlang`, semantic actions are imperative `Java` code written directly by the language developer, and each module is compiled on its own. No compilation step ever sees the full set of attributes written and read by the composed language, so `Neverlang` implements grammar attributes with generic and dynamic maps instead of generating specific data structures for its AST. While this solution greatly improves the composability among artifacts thanks to loose coupling, the absence of a complete specification at compile time denies any compile-time composition correctness guarantee. This reduces the robustness of programming languages developed this way and increases the likelihood of defects in the composition to go unnoticed and become evident only at runtime of the compiler. The well-definedness of AGs, *i.e.*, the guarantee that every attribute of every syntax tree can be computed, has been a concern since Knuth's original paper [61], whose test for circular definitions was later corrected [62]. Decades later, de Moor et al. [32] still pointed out that "the use of an attribute that has not been defined" is a common source of runtime errors in AG-based systems—a problem that could be mitigated by an appropriate type system or static analysis (§2.2). Well-definedness analyses address this problem by verifying that a defining equation exists for every attribute that may be demanded during evaluation. For instance, the modular analysis of Silver [57] performs this check on each language extension in isolation, while JastAdd [37, 49] weaves all the aspects of a specification together before generating its evaluator. As we discuss in §6, these guarantees concern the existence of equations and do not extend to the internal control flow of the (potentially imperative) code implementing each equation, which is exactly the gap this paper addresses.

With this in mind, in the `Neverlang` world, an attempt had already been made by developing a type system and its type inference algorithm [23]. This approach suffered from two main limitations: (i) separate compilation was not supported, and (ii) it was not fully modularizable. These issues arose because, in both cases, it needed to see the entire composition of language constructs. It is therefore of particular interest to determine to which extent the runtime errors which can arise in a `Neverlang`-based project can also be detected statically, without losing the separate compilation constraint and without tightening the coupling among language constructs. Thus, the development of a tool that performs such static checks and warns developers about potentially erroneous runtime instructions becomes a natural objective. With such a tool, we believe language workbenches can achieve the best of both worlds by keeping a loose level of coupling while maintaining static correctness without relaxing the separate compilation constraint of the language extension problem.

In this work, we present a generic approach to achieve this goal based on data-flow analysis and an implementation thereof for the `Neverlang` language workbench, named `nlgcheck`. Concretely, `nlgcheck` is run once separately compiled language components have already been composed into a single system, at which point it shifts the question from "is the attribute defined somewhere in the composition?"—the structural guarantee already offered by the compiler—to "is the attribute initialized before use along every possible execution path of the composed system?", a property no compiler check can currently offer under separate compilation. We show that `nlgcheck` can bridge the gap with the static analysis capabilities offered by compilers without separate compilations by navigating a representation of all possible ASTs that can be generated for a given grammar. Moreover, since it follows the control flow of the imperative code implementing each semantic action, our analysis also detects attributes that are written only within some branches of a conditional statement and then read elsewhere, a case that would otherwise surface as a null pointer exception

at runtime. We do not claim that this makes it more precise than classical well-definedness analyses on AGs specified with equations: its purpose is to extend their guarantees to imperative semantic actions composed after separate compilation (§6). Finally, we complement our contribution with an experiment performed on existing `Neverlang`-based projects, showing that `nlgcheck` could find 73% of injected naming errors over five different Neverlang languages through mutation testing.

The remainder of this paper is structured as follows. Section 2 provides the necessary background, introducing attribute grammars and program dependence analysis [7, 85], while Section 3 introduces the `Neverlang` language workbench and the `Soot` analysis framework that underpin our approach. Section 4 presents our main contribution: a generic, data-flow-based approach for statically checking attribute usage in the context of language composition with separate compilation, detailing the theoretical underpinnings and the rationale behind our method. Concretely, the artifact this analysis is performed on is not the `Neverlang` source specification but the *compiled bytecode* of each semantic action, from which a *control-flow graph* (CFG) is extracted (§4.2). The section then explains how the CFGs of individual semantic actions are composed into the CFG of an entire role (§4.3), how this CFG is visited to detect attribute-access violations while staying sound in the presence of unbounded recursion (§4.4), the optimizations that make this visit practical on real-world languages (§4.5), and, finally, how developers can act on the violations it reports (§4.6). Section 5 evaluates the effectiveness of `nlgcheck` through experiments on real-world `Neverlang`-based projects, including a mutation testing campaign to assess its error-detection capabilities. Section 6 discusses related work, comparing our approach to existing solutions. Finally, Section 7 concludes the paper, summarizing our contributions and outlining directions for future research. An appendix provides additional technical details and supporting material.

2 Background

2.1 Attribute Grammars

A formal extension of context-free grammars is the concept of *attribute grammars* (AG), introduced by Knuth [61] as a way to integrate semantics into the syntactic structure of a language. In his seminal paper "Semantics of Context-Free Languages", the productions of a grammar are enriched with attributes and semantic equations that evaluate these attributes based on parse tree context—enabling systematic semantic analysis and evaluation within grammar-based systems. Attributes allow the transmission of information between different parts of the AST. Just as parsing strategies are typically divided into *bottom-up* and *top-down* approaches, attributes are categorized according to the direction in which their values propagate: *synthesized* or *inherited* [1, 63], respectively. To formally define the attributes, let $G = \langle \Sigma, N, S, \Pi \rangle$ be a context-free grammar, where

- Σ is a set of terminal symbols,
- N is a set of non-terminal symbols,
- $S \in N$ is the distinguished start symbol, and
- Π is a set of productions of the form $A \longrightarrow \alpha$, where $A \in N$ and $\alpha \in (N \cup \Sigma)^*$.

Synthesized Attributes. A *synthesized attribute* is an attribute whose value is computed from the values of the attributes of its children nodes and propagated upward in the tree. Formally, if $A \longrightarrow \alpha_1 \ldots \alpha_n \in \Pi$ is a production, then a synthesized attribute s of A can be defined as:

$$A.s = f(\alpha_{j_1}.s_1, \alpha_{j_2}.s_2, \ldots, \alpha_{j_m}.s_m)$$

where f is a function that computes the value of $A.s$ based on the synthesized attributes $s_1, s_2, \ldots, s_m$ of its children nodes $\{\alpha_{j_1}, \alpha_{j_2}, \ldots, \alpha_{j_m}\} \subseteq \{\alpha_1, \alpha_2, \ldots, \alpha_n\}$.

Inherited Attributes. As stated by Knuth [63], P. Wegner introduced the *inherited attribute* concept. Its value is computed from the values of the attributes of its parent and/or sibling nodes—either synthesized or themselves inherited—and propagated downward in the tree. Formally, for a production $A \longrightarrow \alpha_1 \dots \alpha_n \in \Pi$, an inherited attribute i of α_k $(1 \leq k \leq n)$ can be defined as:

$$\alpha_k.i = g(A.i_1, \dots, A.i_p, \alpha_{l_1}.a_1, \dots, \alpha_{l_q}.a_q)$$

where g is a function that computes $\alpha_k.i$ based on the inherited attributes $i_1, \dots, i_p$ of the parent node A and the attributes $a_1, \dots, a_q$—each either synthesized or inherited—of some sibling nodes $\{\alpha_{l_1}, \dots, \alpha_{l_q}\} \subseteq \{\alpha_1, \dots, \alpha_n\} \setminus \{\alpha_k\}$.

Over the years, several comprehensive works have shaped the study and application of AGs. Early collections such as "Attribute Grammars: Definitions, Systems and Bibliography" provided formal foundations, implementation frameworks, and an extensive bibliography [36]. A comprehensive survey by Paakki [76] classified modern extensions of attribute grammars—structured, modular, object-oriented, logical, and functional variants—and reviewed techniques for parallel and incremental evaluation. In parallel, foundational research addressed decidability and efficiency issues. Jazayeri et al. [52] established the inherent exponential complexity of circularity tests, while later work by Deransart et al. [35] developed more efficient algorithms to improve circularity checking and ensure consistent attribute evaluation. More recently, research on AGs has continued along orthogonal directions: Ringo et al. [81] propose *nanopass attribute grammars*, decomposing a single AG into a sequence of small, independently verifiable transformation passes, which shares our own goal of enabling modular, incremental verification of otherwise monolithic language specifications.

2.1.1 Reference Attribute Grammars. A major limitation of canonical AGs is their handling of *non-local dependencies*, i.e., situations in which a property of one syntax tree node depends on properties of nodes arbitrarily far away in the tree [48]. Several extensions to AGs have been proposed to address this limitation from complementary angles. Hedin [47]'s *door attribute grammars* let syntax-tree attributes hold references to other nodes, so a node's attributes become directly reachable from arbitrarily far away, without traversing the intervening tree structure; Boyland [13] instead studies which non-local dependencies such a grammar exposes, statically and without requiring a full evaluation.

Once attributes can reference arbitrarily distant nodes, classical evaluation orders no longer directly apply: Hoover [51], Johnson and Fischer [54, 55], and Kaiser [56] address this complementary evaluation problem, in particular for the incremental re-evaluation required by interactive language-based editors. Beshers and Campbell [11] and Vorthmann [93] apply this line of work to two concrete sources of non-local information—state that persists across incremental edits, and name binding/visibility in object-oriented languages, respectively—before Hedin [48] unified and generalized these ideas into *reference attribute grammars*. The need for such extensions arises from the fact that many semantic analyses, such as *name resolution* and *type checking*, inherently involve non-local dependencies. Reference attribute grammars address this by allowing attributes to hold references to other nodes in the syntax tree, enabling direct access to their attributes regardless of their position in the tree. This capability is particularly useful for implementing complex language features, such as *scoping rules* and *inheritance hierarchies* in object-oriented languages.

2.2 Program Dependence Analysis

Foundational to many static program analyses is the concept of *program dependence analysis*. The main seminal work in this area are Towle [85]'s PhD thesis, Banerjee [7], and Allen [5], and the papers by Wolfe [97] and Banerjee [8]. This analysis identifies dependencies between different parts of a program, that is, a statement A is said to *depend* on another statement B if the execution

of A requires information from B. These dependencies can be classified into two main categories: *control dependencies* (§2.2.2) and *data dependencies* (§2.2.3).

2.2.1 Dominance. F. Allen [4] introduced the concept of *control flow graph* (CFG) as a way to represent all possible paths that a program can take during its execution. Formally, a CFG is a directed graph $G = (V, E, e_y, e_t)$, where:

- V is a set of vertices (or nodes) representing basic blocks of code,
- $E \subseteq V \times V$ is a set of edges representing a control flow between nodes,
- $e_y \in V$ is the entry node, and
- $e_t \in V$ is the exit node.

Since E is a set of pairs of nodes rather than a multiset, G admits no parallel edges: at most one edge connects any given ordered pair of nodes, so a path can be identified unambiguously with the sequence of nodes it visits. In a CFG, a node A is said to *(pre)dominate* another node B if every path from e_y to B must pass through A. Formally, A dom B *iif*:

$$\forall P \in \text{Paths}(e_y, B),\ A \in P$$

where $\text{Paths}(e_y, B)$ is the set of all paths from e_y to B, each represented as the sequence of nodes it visits. The *strictly dominates* relation (A $\overline{\text{dom}}$ B) is defined similarly, but it requires that $A \neq B$. The *immediate dominator* of a node B is the unique node A such that A strictly dominates B and does not strictly dominate any other node that also dominates B. Every node has an immediate dominator, except e_y [69]. We write A idom B to denote that A is the immediate dominator of B. Intuitively, A is the last required "checkpoint" before reaching B in the control flow. Formally, A idom B *iif*:

$$A\ \overline{\text{dom}}\ B \wedge \forall C \in V : (C\ \text{dom}\ B \wedge C \neq A) \Rightarrow \neg(A\ \overline{\text{dom}}\ C).$$

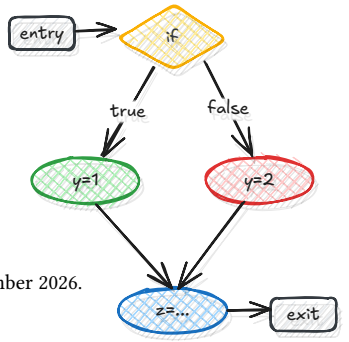
The *dominance frontier* of a node A is the set of all nodes B such that A dominates an immediate predecessor of B, but A does not strictly dominate B. Formally, the dominance frontier of A is defined as:

$$\text{DF}(A) = \{B \in V \mid \exists C \in \text{Pred}(B) : A\ \text{dom}\ C \wedge \neg(A\ \overline{\text{dom}}\ B)\}$$

where $\text{Pred}(B)$ is the set of immediate predecessors of B in the CFG. Intuitively, the dominance frontier of A is the set of nodes where the control exercised by A ends. In the Listing 1, the dominance frontier of the node `y=1` is the set { `z=...` } because: (1) the node `y=1` dominates a predecessor of `z=...` (itself), and (2) the node `y=1` does not strictly dominate `z=...` (since `y=2` is another predecessor of `z=...` that is not dominated by `y=1`).

Dominator trees are a tree-like structure rooted at e_y, where each node A has a child B if A is the immediate dominator of B. The *post-dominance* relation is the dual of dominance: A dom_{post} B *iif* $\forall P \in \text{Paths}(B, e_t), A \in P$, and A *strictly post-dominates* B (A $\overline{\text{dom}}_{post}$ B) if, in addition, $A \neq B$. *Dominance* was first introduced by Prosser [78] in a 1959 study on the analysis of flow diagrams. However, Prosser did not provide an algorithm for computing dominance. A practical method for its computation was proposed ten years later by Lowry and Medlock [71]. Lengauer and Tarjan [69] developed a more efficient algorithm for computing dominator trees in *almost* linear time by using *union-find* with path compression, improving existing solutions [3, 79], and this algorithm was then adopted by the GNU Compiler Collection (GCC). More than

```
if (x > 0)
    y = 1;
else
    y = 2;
z = y + 1;
```



Listing 1. Dominance frontier.

twenty years later, Cooper et al. [27] presented a simpler algorithm that is faster in practice, relying on the *reverse post-order* traversal of the CFG. The Low Level Virtual Machine (LLVM) compiler infrastructure, on the other hand, uses the algorithm by Georgiadis et al. [45], which has a worst-case time complexity of $O(n^2)$ but is more efficient in practice and, additionally, supports incremental updates [44].[4]

2.2.2 Control Dependencies. *Control Dependencies* are post-dominance frontier in the CFG [29]. They refer to the dependencies that arise from the control flow of a program, such as conditional statements and loops. A statement A is said to be control dependent on another statement B if the execution of A depends on the outcome of B. Formally, following Cytron et al. [30], $A \xrightarrow{c} B$ if both of the following conditions hold:

$$\exists P \in \text{Paths}(B, A) : \forall C \in P, C \neq B \Rightarrow A \ \text{dom}_{post}\ C,$$

$$\neg(A \ \overline{\text{dom}}_{post}\ B).$$

That is, A post-dominates every node that follows B along some path from B to A, but does not strictly post-dominate B itself. When $A \neq B$, the second condition means that some path in $\text{Paths}(B, e_t)$ avoids A, so the outcome of B decides whether A is executed. Note that the arrow in $A \xrightarrow{c} B$ points from the dependent statement to the one it depends on, and therefore runs in the opposite direction to the corresponding CFG edge, along which control actually flows from B to A: in Listing 2, the CFG edge goes from the condition `a < 0` to `x = -1`, executed only if the condition holds, while the control-dependency arrow is written `x = -1` $\xrightarrow{c}$ `a < 0`, read as "`x = -1` is control dependent on `a < 0`". Due to the impact of data dependencies on control dependencies, the two concepts are often intertwined. J. Allen et al. [6] show how control dependencies can be transformed into data dependencies using the *if-conversion* technique. This approach eliminates conditional execution by evaluating both control-flow paths unconditionally and then using a predicate to select the appropriate result, which is finally assigned to the target variable. Listing 2 illustrates a simple example of control dependency, where the statement `x = -1` is control dependent on the if-statement.

2.2.3 Data Dependencies. Banerjee [8] formalized the concept of *data dependencies* within the context of program analysis. Data dependencies arise when the execution of one statement depends on the data produced by another statement. *Data hazards* are a specific type of data dependency that can occur in pipelined processors, where the execution of one instruction depends on the result of a previous instruction that has not yet completed; they can lead to *race conditions*. Three main types of data hazards are identified: *read-after-write* (RAW), *write-after-read* (WAR), and *write-after-write* (WAW). In accordance with Bernstein [9], by contrast, data dependencies can be classified into: (1) *(true)flow dependencies*, (2) *anti-dependencies*, and (3) *output dependencies*. In the following, we use the notation s_i to refer to the i-th statement in the Listing 3.

Flow Dependency. A statement s_3 is said to be *flow dependent* on another statement s_2 (denoted by $s_3\ \delta^f\ s_2$) if s_3 uses a value produced by s_2. RAW hazards occur when a violation of this dependency happens. In Listing 3, s_3 (`int b = a + c;`) is flow dependent on s_2 (`int a = 1;`), since it uses the value of `a` that s_2 produced.

Anti-Dependency. An *anti-dependency* takes place when a statement s_4 writes to a variable that is read by another statement s_3 (denoted by $s_4\ \delta^a\ s_3$). This can lead to WAR hazards if the order

[4]For a recent empirical characterization of LLVM's broader optimization pipeline, see Bruzzone and Cazzola [14].

```
1 int f(int a) {
2     int x = 1;
3     if (a < 0) {
4         x = -1;
5     }
6     return x;
7 }
```

Listing 2. Control dependency.

```
1 int c = 0;
2 int a = 1;
3 int b = a + c;
4 c = 2;
5 c = b + 1;
```

Listing 3. Data dependency.

of execution is not preserved. In Listing 3, s_4 (`c = 2;`) is anti-dependent on s_3 (`int b = a + c;`), since s_4 writes to `c`, which s_3 had read: swapping their order would change s_3's result.

Output Dependency. An *output dependency* is symmetric and occurs when two statements s_4 and s_5 write to the same variable (denoted by $s_4\ \delta^o\ s_5$). WAW hazards happens if a violation of this dependency occurs. In Listing 3, s_4 (`c = 2;`) and s_5 (`c = b + 1;`) both write to `c`, so their relative order must be preserved to guarantee `c`'s final value.

An additional type of data dependency is the *input dependency*, which occurs when two statements read from the same variable. Anti-dependencies and output dependencies are often grouped together as *storage-related dependencies* [1, 50]—they may be removed by renaming variables.

3 Software Frameworks

This section introduces the two main software frameworks that underpin the work described in this paper: the `Neverlang` language workbench and the `Soot` static analysis framework. Neverlang provides a modular infrastructure for language engineering, enabling the definition, composition, and extension of programming languages through reusable components. Soot, on the other hand, is a mature and widely used framework for analyzing and transforming Java programs at the bytecode and intermediate representation levels. Together, these tools form the foundation for our approach to integrating advanced program analysis capabilities into language development workflows.

3.1 Neverlang in a Nutshell

The `Neverlang` [20, 25, 86] language workbench[5] promotes code reusability and separation of concerns leveraging the *language-feature* approach.

The basic development unit is the **`module`**, as shown in line 1 of Listing 4. A module may contain a **`reference syntax`** and could have zero or multiple **`role`**s. A role, used to define the semantics, is a composition unit that defines actions that should be executed when some syntax is recognized, as defined by *syntax-directed translation* [2]. Each role corresponds to one compilation phase, such as type checking or evaluation, and groups the semantic actions executed during one traversal of the AST: at runtime, the roles of a language are executed one after the other, each performing its own traversal of the whole tree. Syntax definitions are defined using *Backus-Naur form* grammars, represented as sets of *productions* and *terminals*. Syntax definitions and semantic **`role`**s are tied together using **`slice`**s. Listing 4 shows a simple example of a Neverlang module implementing a backup task of the `LogLang` *language product line*, i.e., a family of related languages sharing a common set of reusable components [16]. Reference syntax is defined in lines 2–6. The *categories*

[5]The term *language workbench* was coined by Martin Fowler in his 2005 article "Language Workbenches: The Killer-App for Domain Specific Languages" [42]. They are software tools that provide an integrated environment for designing, implementing, and maintaining programming languages. Language workbenches typically offer features such as syntax definition, semantic analysis, code generation, and editing support.

```
1  module Backup {
2    reference syntax {
3      BackupCmd: Backup ⇐ "backup" String  String;
4      BackupAsCmd: Cmd ⇐ Backup;
5      categories  : Keyword = { "backup" };
6    }
7    role(execution) {
8      0 .{
9          String src  = $1.string, dest = $2.string;
10         $$FileOp.backup(src, dest);
11     }.
12   }
13 }
14 slice BackupSlice {
15   concrete syntax from Backup
16   module Backup with role execution
17   module BackupPermCheck with role permissions
18 }
19 language LogLang {
20   slices BackupSlice RemoveSlice RenameSlice
21          MergeSlice Task Main LogLangTypes
22   endemic slices FileOpEndemic PermEndemic
23   roles syntax < terminal-evaluation
24                < permissions
25                : execution
26 }
```

Listing 4. Syntax and semantics for the backup task.

(line 5) are used to generate the syntax highlighting for the *integrated development environments* (IDEs).

A production may be given a *label*, written before it and followed by a colon, such as `BackupCmd` in line 3. The label names the production and is distinct from its left-hand-side nonterminal, `Backup`. Within a role, a semantic action is attached to a nonterminal occurrence of the reference syntax, identified either by its position or through the label of its production. Positions count nonterminals only and are numbered from 0 across the whole reference syntax, from the top left to the bottom right. In Listing 4, the nonterminals of the first production (`Backup` and the two `String`s) have positions 0, 1, and 2, and those of the second production (`Cmd` and `Backup`) have positions 3 and 4. The action in line 8 is attached to position 0, *i.e.*, to the left-hand side of the first production, and is executed whenever a node built by that production is visited. Inside the action, `$0`, `$1`, and `$2` denote the nonterminals of that same production, so `$1.string` reads the `string` attribute of the first `String` child. An action for the second production would be written in the same way, either as `3 .{...}.` or, through its label, as `BackupAsCmd: .{...}.`. A label followed by an index, such as `BackupCmd[1]`, denotes the corresponding nonterminal of the labeled production. Two productions are given rather than one: the production labeled `BackupCmd` defines the concrete syntax of the backup command itself, while the one labeled `BackupAsCmd` (line 4) injects `Backup` as one alternative of the more general `Cmd` nonterminal. Other modules can likewise contribute their own alternatives for `Cmd` (*e.g.*, `Remove`, `Rename`) without altering this module, which is what makes the statement syntax extensible across slices.

Line 10 shows the `$$`-prefixed identifier notation (`$$FileOp`), used to access a singleton instance of a service object—here, a library providing file system manipulation primitives—from within a

semantic action. Such a service is implemented by a separate component called an *endemic slice* (line 22), which declares an interface and a concrete implementation that the `Neverlang` runtime automatically instantiates when the language implementation is initialized. Unlike ordinary **`slice`**s, an endemic slice does not contribute any syntax or AST-visiting role of its own: it only provides a runtime service that semantic actions may invoke directly, independently of the specific production or attributes they are attached to, and is therefore no less compositional than any other module.

The order in which roles are executed is declared by the **`roles`** clause of the language, as shown in lines 23–25 and detailed below. Besides the ordering, each role is also associated with an *execution mode*, which determines how the role traverses the AST. In *postorder* mode, the default, the children of each node are visited before the action attached to the node itself, without requiring the semantic actions to trigger the traversal explicitly. In *manual* mode, selected by prefixing the role name with `+` in the **`roles`** clause (*e.g.*, `<+ evaluation`), the action attached to a node is executed first and is itself responsible for visiting the children, through the `eval` construct (§4), while nodes without an action are still traversed automatically. Unlike the visit sequences of Kastens' *ordered attribute grammars* (OAGs) [59], which an OAG system *infers* from the attribute dependency graph and are guaranteed by construction to evaluate every attribute after its dependencies, `eval` in manual mode is an explicit control-transfer statement written by the language developer: `Neverlang` performs no such inference and does not check that the developer's chosen order actually respects attribute dependencies. This is precisely the gap that `nlgcheck` is designed to close for manual-mode roles: a static, after-the-fact well-definedness check standing in for the guarantee that an OAG's visit-sequence construction would otherwise have provided at generation time. In contrast, the `BackupSlice` (lines 14–18) reveals how the syntax and semantics are tied together; choosing the **`concrete syntax`** from the Backup module (line 15), and two **`role`**s from two different modules (lines 16–17). Finally, the **`language`** can be created by composing multiple **`slices`** (line 20). The composition in Neverlang is twofold: between modules and between slices. Thus, the grammars are merged to generate the complete language parser. On the other hand, the semantic actions are composed in a pipeline, and each **`role`** traverses the syntax tree in the order specified in the **`roles`** clause (lines 23–25). The `<` operator states that the role on its left must run to completion before the role on its right starts (*e.g.*, **`syntax`** before `terminal-evaluation` before `permissions`). The `:` operator, instead, combines roles into a single *layered* traversal: with `permissions : execution`, each node executes its `permissions` action immediately followed by its `execution` action, so the two roles are interleaved node by node instead of requiring two separate traversals. Beyond slices, Neverlang also supports *bundles*: a bundle package is a separately compiled sublanguage (i.e., a self-contained group of modules that does not necessarily define a complete language on its own) that can be shipped and reused as a single unit across different host languages, without requiring access to its source form.

Since 2019 [65], Neverlang has always had an interest in providing the *editing support* for the languages developed with it. Initially, the editing support was provided through the *Eclipse IDE* plugin, which was recently replaced by the *de facto* standards *language server protocol* (LSP) and *debug adapter protocol* [15]. Both protocols decouple editing/debugging support from any specific editor: any editor that implements the client side of the protocol (e.g., Visual Studio Code, the Eclipse IDE, or Neovim) can provide editing and debugging support for a language, provided the language itself ships a compatible language server. To provide LSP services, Neverlang leverages the `Typelang` DSL—a language for type systems definition—to generate *language servers* that can be used by any editor that supports the LSP.

3.2 Soot Library

`Soot` [87, 88] is a Java analysis and transformation framework, first released in 1999 and still actively maintained today. Initially conceived for bytecode optimization, it has evolved into a comprehensive tool for static analysis [77, 84, 87]. A recent effort ("SootUp") completely reimplements Soot with a more modular design, modernizing the original tool while preserving its familiar APIs [58].

Soot's main strength lies in its intermediate representations (IR), in particular *Jimple*, a typed three-address code designed to make control flow and data flow explicit and thus simplify static analyses. Other IRs (*Shimple* in *static single assignment* form, *Baf* for bytecode-level rewriting [82], and *Grimp* for decompilation [74]) extend its flexibility [66]. On top of these IRs, Soot offers standard analyses such as call-graph construction [58], pointer analysis (*Spark*) [70], and data-flow analysis frameworks (*Heros*) [12, 80, 83]. These enable both whole-program analyses and program transformations, with outputs ranging from optimized bytecode to instrumented programs.

Soot has been applied to a wide range of program analysis and transformation tasks. Early on it was used for Java optimizations and class file instrumentation [31], but it quickly found broad academic use. It has underpinned research in pointer analysis, concurrency analysis, symbolic execution, and the static portions of hybrid analysis techniques [66].

4 Program Dependence Analysis for Separately Compiled Languages

This section presents the concept behind `nlgcheck`, which represents the core of our contribution.

4.1 Notation

We restate here, in a form specialized to the needs of this section, the grammar and attribute notions already introduced in §2.1: a **grammar** is defined as a quadruple (Σ, N, S, Π), in which Σ is a set of terminal symbols (not interesting in this context, since attributes are only attached to nonterminal symbols), N is a set of nonterminal symbols, $S \in N$ is the start nonterminal symbol (which in `Neverlang` is always named `Program` by convention), and Π is a set of production rules.

A **production** $p \in \Pi$ is written in the form $X_0 \leftarrow X_1 \cdots X_n$, with $X_i \in N$ all being nonterminal symbols, and has the meaning that during the parsing process, X_0 (which is called the **left-hand side** of the production) can be replaced by the $X_1 \cdots X_n$ sequence (called the **right-hand side** of the production). The notation $|p|$ is used to indicate the number $n + 1$ of nonterminals forming the production (one on the left-hand side and n on the right-hand side) and $p[i]$ to indicate the i-th nonterminal X_i.[6]

Given a production $p \in \Pi$, we will refer to the attribute named a on the i-th nonterminal of p with the notation $p[i].a$: with $p[0].a$, we indicate an attribute on the left-hand nonterminal of the production, and with $p[i].a$ (with $i \geq 1$), an attribute on one of the nonterminals on the right-hand side.

A **semantic action**, for the scope of this section, can be defined as a sequence of executable code attached to a production, some of which can set or access attributes associated with the production's nonterminals.[7]

Any attribute set or accessed by an instruction has a **type**, which is usually known during the compilation of the semantic action code. We call T the set of all available types. T is partially ordered since, given two types $t_1, t_2 \in T$, exactly one of the following is true:

- t_1 and t_2 are the same type ($t_1 = t_2$);

[6]Note that this is a simplification of the definition of production, in which the right-hand side is a sequence that can contain both terminal and nonterminal symbols. However, this does not pose a limitation in this context, since our approach focuses on the analysis of attributes attached to nonterminals.

[7]This is also a simplification, but it suffices to discuss anything relevant to the problem we are dealing with.

- t_1 is a subtype of t_2 ($t_1 <: t_2$);
- t_1 is a supertype of t_2 ($t_1 :> t_2$);
- none of the above (t_1 and t_2 are unrelated).

For brevity, we write $t_1 \leq: t_2$ (*subtype-or-equal*) to mean $t_1 <: t_2 \vee t_1 = t_2$. A value of type t_1 is *assignable* to a location of type t_2 exactly when $t_1 \leq: t_2$.

To refer to the type of an attribute $p[i].a$, we use the notation $\tau(p[i].a)$. More precisely, given $\mathbb{N}$ as the set of natural numbers and I as the set of valid attribute names,[8] we can define τ as a partial function $\tau : \Pi \times \mathbb{N} \times I \rightarrow T$ that maps the triplet (production, index of a nonterminal in the production, and attribute name) to the type of the corresponding attribute. $\tau(p[i].a) \coloneqq \tau(p, i, a)$ is not defined if p has less than i nonterminals or if $p[i]$ has no attribute named a. The attribute name a alone does not suffice to identify a type: the same nonterminal can occur at several positions of the same production (*e.g.*, in a right-recursive rule), and different productions can attach an attribute with the same name but a different type to occurrences of the same nonterminal. The triple (p, i, a) disambiguates both cases.

4.2 Verification Model

A simplified model resembling the type-checking algorithms already found in most AG systems (§A) proved insufficient, as detailed there. We therefore opted for a more complete model that closely represents the executable action. The natural choice in this regard is the CFG [4].

The CFG for a semantic action we will be using in this work is an instance of the CFG presented in §2.2.1. We define a directed graph $G = (V, E)$ with $E \subseteq V \times V$ in which every vertex (or node) has a unique identifier (for the sake of simplicity, we will assume $V \subset \mathbb{N}$) and is associated with an *action*, each being a sequence of statements that forms an atomic subset of a more complex semantic action. We also define an **entry node** and an **exit node** of the CFG, representing the first and last action of the semantic action execution; the execution can follow different paths but always begins at the entry node and terminates at the exit node. The entry node and exit node are denoted as $v_{entry}(G)$ and $v_{exit}(G)$, respectively.

The semantics of each node are described through their mapping to an action by means of the $a_{node} : V \rightarrow A$ function, where A is the set of all possible instructions that can be executed in a semantic action. In this work, we consider the following instructions, grouped by category. Most instructions can be parametrized, making A an infinite set. We adopt the following notation for the parameters and their types: $\mathbb{N}$ is the set of natural numbers, I is the set of valid attribute identifiers, and T is the set of types.[9]

Attribute actions. Attribute actions model access and manipulation of AST node attributes. We consider the following attribute actions (and their respective CFG nodes):

$WriteAttr(i, n, t)$, with $i \in \mathbb{N}, n \in I, t \in T$
The action sets an attribute named n of type t on the i-th nonterminal of the production. If $i = 0$, then the attribute is written on the node currently being visited; otherwise, it is written to its i-th child.

Unlike classical AGs, where the single-assignment rule guarantees that each attribute instance is written by exactly one equation, our model treats attributes as ordinary mutable locations: since semantic actions are imperative code rather than declarative equations, nothing in `Neverlang` prevents a semantic action from executing several $WriteAttr$ nodes for the same attribute along

[8] Also called *identifiers*; for all intents and purposes, we can assume that the set of valid identifiers is a subset of all ASCII strings.

[9] For the purposes of this work, T coincides with the set of all the types in the `Java` Virtual Machine (JVM).

the same path (*e.g.*, inside a loop, or across the branches of a conditional that both assign it before continuing). $WriteAttr(i, n, t)$ therefore always overwrites any previous value, and a *ReadAttr* is checked against the type of the most recent write reaching it along the current path—ordinary imperative dataflow semantics, not the AG single-assignment discipline.

$ReadAttr(i, n, t)$, with $i \in \mathbb{N}, n \in I, t \in T$

The action accesses an attribute named n from the i-th nonterminal of the production and casts its value to type t. Please note that the previous restriction about the ability to read attributes only from child nodes and write only to the current node (see §A) is lifted, as this model does not assume postorder execution.

$CopyAttr(i_d, n_d, i_s, n_s)$, with $i_d, i_s \in \mathbb{N}, n_d, n_s \in I$

The action accesses an attribute named n_s from the i_s-th nonterminal of the production and sets an attribute named n_d on the i_d-th nonterminal of the same production to the retrieved value.[a] The runtime behavior of $CopyAttr$ is identical to performing a $ReadAttr(i_s, n_s, t)$ followed by a $WriteAttr(i_d, n_d, t)$ with the same t and value, but the distinction matters in this context. Copying an attribute does not involve interacting with its type, so the type is preserved even when not explicitly mentioned. Using the read-write sequence instead, without that type information, would default to $t = Object$, losing relevant type information.

[a]The s and d subscripts stand for *source* and *destination*.

$CheckAttrType(i, n, t)$, with $i \in \mathbb{N}, n \in I, t \in T$

The action accesses an attribute named n from the i-th nonterminal of the production and compares its type to t.[a] This action does not modify the state nor can it cause runtime errors, but it is used to raise warnings: using **`instanceof`** on attribute values is considered an anti-pattern in `Neverlang`, as the same check could be performed more idiomatically with dedicated type guards.

[a]in `Java`, this is done with the **`instanceof`** operator; similar or equivalent type tests are available in other languages.

$PropagateAttrs()$

Assuming that the semantic action is attached to a production with a single nonterminal on the right-hand side, thus has the form $X_0 \leftarrow X_1$, each attribute present on X_1 is copied to X_0 with the same name and type, unless already present. The code that performs such a copy is not written by the developers. Instead, the copy is performed automatically by the `Neverlang` runtime. Thus, we attach additional nodes to the CFG representing this implicit behavior. Like $CopyAttr$, and for the same reason, $PropagateAttrs$ does not perform any type checking: since every propagated attribute retains its original type, no mismatch can occur.

Evaluation actions. Evaluation actions model the control that the semantic action has over the tree-traversal order. As exemplified in Listing 5 in the context of the implementation of the ternary operator language feature, a semantic action can contain the `eval $i` construct (lines 7, 9, 12) to instruct the `Neverlang` runtime to suspend the execution of the semantic action, instead proceeding by evaluating the i-th child of the current AST node according to the same role. This particular example is interpreter-style, in that the semantic action directly computes and returns the expression's value (`$0.val`) rather than emitting machine code for it. Our model, however, only concerns itself with which attributes a semantic action reads and writes, and the order in which it does so, regardless of whether the action's ultimate effect is to compute a value or to emit code that will compute it at a later stage—the same reasoning therefore applies unchanged to a semantic action implementing, *e.g.*, `LLVM` IR code generation for the same ternary operator. Once the `eval` is complete, the execution of the current semantic action is resumed (as is typically done when invoking functions in most programming languages). We consider the following attribute actions (and their respective CFG nodes):

```
module componentlib.clike.expressions.TernaryOp {
  reference syntax {
    Op: Expression <- Expression "?" Expression ":" Expression ;
  }
  role (evaluation) {
    0 .{
      eval $Op[1];
      if ((Boolean) $Op[1].val) {
        eval $Op[2];
        $Op.val = $Op[2].val;
      } else {
        eval $Op[3];
        $Op.val = $Op[3].val;
      }
    }.
  }
}
```

Listing 5. `Neverlang` code implementing a `C`-like ternary expression language feature.

BeginEvalMetaAction(i), with $i \in \mathbb{N}$

The action evaluates the i-th nonterminal of the production, according to the role currently being executed.

EndEvalMetaAction()

The action is resumed after visiting another node.

Concretely, these two nodes always appear jointly in the CFG: a node mapped to a *BeginEvalMetaAction* always has exactly one outgoing edge, connected to an *EndEvalMetaAction*, which is its only incoming edge. The operation is atomic and spans over two nodes for purely practical CFG manipulation reasons (please refer to the next section for further details). `Neverlang` provides syntactic sugar for common behaviors, such as making a single semantic action behave as if it were used in a postorder role: this is denoted with the `@{...}.` operator. Such syntactic sugar does not impact the creation of the CFG, as it is equivalent to performing `eval` statements for each of the right-hand side nonterminals of the production before proceeding further.

Other actions. Other actions unrelated to previous categories are the following:

Nop()

The action does not represent any real operation; thus, the node is within the CFG for structural reasons (such as denoting branch points).

IgnoreBranchAction()

The action throws a runtime exception that would interrupt the execution of the interpreter/compiler. Since the visit cannot proceed further after throwing an exception, the branch is ignored.

Fig. 1 shows the CFG obtained from the semantic action presented in Listing 5, according to the concepts presented thus far. The label on top of Fig. 1 lists the production and the role to which the semantic action is attached. Each node is uniquely identified with a hexadecimal number. For the implementation, the CFG for a semantic action is constructed by analyzing the compiled bytecode of the `Java` classes generated by the `Neverlang` compiler using `Soot`, as detailed later in this section.

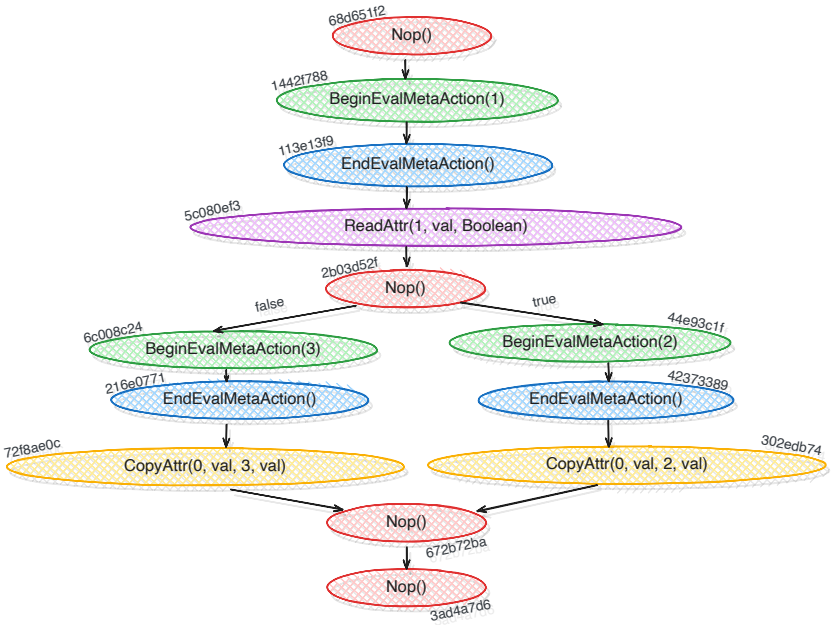


Fig. 1. Control flow graph generated by `nlgcheck` for the semantic action shown in Listing 5.

4.3 Modeling Roles

The concept of a CFG is not limited to individual semantic actions, equivalent to *intra-procedural control-flow graphs* in traditional CFGs.

Inspecting the behavior of the whole compiler/interpreter, thus simulating the entire AST visit (role, in `Neverlang`), requires taking inspiration from static analysis techniques based on the *inter-procedural control-flow graph* instead [19]. The natural choice is to combine the CFGs for the individual semantic actions—connecting their entry, exit, and eval nodes—into a unique CFG for the entire role through adequate edges.

Such information is obtained from the language grammar—*e.g.*, the semantic action for `Stmt ⇐ "if" Expr "then" Stmt`. To indicate that the execution flow may continue towards a semantic action whose production's head is the `Expr` nonterminal a $BeginEvalMetaAction(1)$ node is used—*i.e.*, a production in the `Expr ⇐ ...` form; then, the semantic actions can be connected accordingly.

There are a few factors to consider when navigating the CFG built in this way.

Returning to the correct call site. In general-purpose program analysis, a given procedure may potentially be called by many others (i.e., it has many *call sites*). In our model, each semantic action equates to a procedure. Thus, the call site of a semantic action G attached to a production $p : Y \leftarrow \cdots$ may reside in any of the G_c semantic actions (where the c subscript denotes the *call site*) attached to productions p_c having Y on their right-hand side.

Moreover, if p_c has multiple instances of Y on the right-hand side, then the call site of G may correspond to each instance of Y in p_c. Let us consider a "caller" semantic action $G_i = (V_i, E_i)$ attached to a production p_i and a "called" semantic action $G_j = (V_j, E_j)$ attached to a production p_j. We take $v_{beginEval}, v_{endEval} \in V_i$ and $k \in \mathbb{N}$ such that

$$p_i[k] = p_j[0],$$

$$a_{node}(v_{beginEval}) = BeginEvalMetaAction(k),$$
$$a_{node}(v_{endEval}) = EndEvalMetaAction(),$$
$$(v_{beginEval}, v_{endEval}) \in E_i$$

Then, given the edge set E_r of the CFG G_r, the control flow being transferred from G_i to G_j is performed as follows:

(1) remove $(v_{beginEval}, v_{endEval})$ from E_r (as the execution of G_j cannot be skipped);
(2) add $(v_{beginEval}, v_{entry}(G_j))$ to E_r;
(3) add $(v_{exit}(G_j), v_{endEval})$ to E_r.

Since the language grammar potentially comprises many other productions $p_l \neq p_i$ such that $\exists k' : p_l[k'] = p_j[0]$, these steps are performed on each $p_i, p_l, \ldots$, leading to several pairs of vertices in the form $v'_{beginEval}, v'_{endEval} \in V_l \subseteq V_r$ such that $(v'_{beginEval}, v_{entry}(G_j)) \in E_r$ and $(v_{exit}(G_j), v'_{endEval}) \in E_r$. Some of these vertices may never be navigated in a real interpreter; thus, we introduce *tagged entry* and *exit edges*. We define a set L of tags, two sets $E_{entry}, E_{exit} \subset E_r$ such that $E_{entry} \cap E_{exit} = \emptyset$ and a function $l_{edge} : (E_{entry} \cup E_{exit}) \to L$. We define a *tagged entry edge* as an edge $e \in E_{entry}$ such that $l_{edge}(e) = l$ (with $l \in L$) and a *tagged exit edge* as an edge $e \in E_{exit}$ such that $l_{edge}(e) = l$. Then, we can imagine paths as well-parenthesized expressions, considering valid only those paths in which every exit edge (closing bracket) has a corresponding entry edge (opening bracket) with the same tag.

This leads to a formal definition for a well-formed CFG path:

- the empty path ϵ is a well-formed path;
- given a pair of vertices $v_a, v_b \in V_r$ connected by an untagged edge $e = (v_a, v_b)$, such that $e \in E_r$ and $e \notin (E_{entry} \cup E_{exit})$, $v_a v_b$ is a well-formed path;
- given the vertices $v_a, v_b, v_c, v_d \in V_r$ and a well-formed path $w = v_b \ldots v_c$, let $e_{entry} = (v_a, v_b)$ and $e_{exit} = (v_c, v_d)$; then $v_a w v_d$ is a well-formed path if

$$e_{entry} \in E_{entry} \wedge e_{exit} \in E_{exit} \wedge l_{edge}(e_{entry}) = l_{edge}(e_{exit})$$

- given two well-formed paths w_a and w_b, their concatenation $w_a w_b$ is a well-formed path.

Finally, any prefix of a well-formed path is a *valid path*.

For instance, reusing the nested-ternary tagged entry/exit edges introduced in §4.4.7—entry edge (`44e93c1f`, `68d651f2`) and exit edge (`3ad4a7d6`, `42373389`), both tagged l—a path that traverses the entry edge, visits the whole callee CFG, and returns via that same exit edge is well-formed (the tags match, as required by the third item above). By contrast, a path that traverses the entry edge and then simply stops partway through the callee, without ever traversing a corresponding exit edge tagged l, is a *valid* path (it is a prefix of a well-formed one, namely the path that does go on to complete the call) but is not itself well-formed, since it is not balanced.

Two semantic actions G_i and G_j are therefore connected as follows:

(1) remove $(v_{beginEval}, v_{endEval})$ from E_r;
(2) add $e_1 = (v_{beginEval}, v_{entry}(G_j))$ to E_{entry} (and thus to E_r);
(3) add $e_2 = (v_{exit}(G_j), v_{endEval})$ to E_{exit} (and thus to E_r);
(4) let l be a new tag ($l \notin L$) and add it to L;
(5) let $l_{edge}(e_1) = l_{edge}(e_2) = l$.

When the visit reaches $v_{exit}(G_j)$, only one among many outgoing edges matches the tag followed when entering the semantic action. Following such an edge returns control to the correct call site.

Context switching. Because a semantic action may, through `eval`, transfer control to another semantic action that (directly or through a chain of further calls) transfers control back to a semantic action attached to the same production—as illustrated by the recursive ternary-expression

example of §4.4.7—simulating execution requires keeping track of a possibly unbounded number of nested, concurrently active invocations. This calls for two stack data structures: one to store the production context of each active invocation (introduced in this section) and one to track the nesting of tagged entry/exit edges as calls are made and returned from (§4.4.7). This bookkeeping is a direct consequence of `eval` being an explicit, developer-written control transfer rather than an OAG-style inferred visit sequence (§3.1): an OAG evaluator's generated visit sequence needs no such simulation, since it is correct by construction, whereas here nothing guarantees a priori that a chain of `eval` calls respects attribute dependencies, which is exactly what the stacks introduced below let us check. Execution simulation involves keeping the state in proper data structures. The state comprises pieces of information that will be detailed later; for now, consider that the behavior of a semantic action depends on contextual information. In fact, several of the actions introduced earlier involve i parameters that refer to the AST node being visited. Thus, we extend the model with two more actions: $EnterCtx(i)$ and $LeaveCtx()$ (with $i \in \mathbb{N}$). The semantics are akin to procedures in low-level programming, which often involve a *prologue* and an *epilogue*: additional instructions surrounding the actual body, often used to allocate and free stack frames. Likewise, we can use a stack data structure: each item on the stack, which we call a *production context*, stores information (name and type) of the attributes of the current node's children, with the base of the stack representing the root AST node, which can also store its own attributes.

When the current semantic action sets or accesses an attribute on a right-hand-side nonterminal ($i \geq 1$) of its attached production, the operation is performed directly on the production context on top of the stack. When the current semantic action sets or accesses the left-hand-side nonterminal ($i = 0$), the operation is performed on the second-to-top production context of the stack with position i_c. This model reflects how attributes are passed at runtime between semantic actions depending on their position within the productions.

$EnterCtx(i)$ and $LeaveCtx()$ are therefore equivalent to procedure prologues and epilogues, respectively: the former pushes a new production context with $i_c = i$ to the stack, while the latter pops the production context from the top of the stack. These two actions are not additional nodes inserted alongside *BeginEvalMetaAction*/*EndEvalMetaAction* (§4.3); rather, once the composition step described there connects a caller to a callee through a tagged entry/exit edge pair, the node mapped to $BeginEvalMetaAction(k)$ is remapped to $EnterCtx(k)$, and the node mapped to $EndEvalMetaAction()$ is remapped to $LeaveCtx()$. They are the same two nodes, in the same CFG positions, simply carrying different actions once their role in an actual call is resolved.

This recursive call structure is unrelated to the classical notion of *circular attributes*: since `nlgcheck` only performs a static analysis over CFG paths, rather than actually evaluating attributes, it is not concerned with whether a fixed-point computation over attribute values would converge, but only with whether every read is preceded by a matching write along a given path. The depth-2 cutoff of §4.4.8 accordingly bounds the recursive unfolding of the *analysis*, not of any attribute value computation. Exceptions raised by semantic actions are not prohibited: they are already accounted for by the $IgnoreBranchAction()$ node (§4.2), which marks the branch as an unreachable dead end for the purposes of the CFG traversal, since no further attribute accesses can occur once control has left the current semantic action via an exception.

CFG completion. Entry and exit nodes must be defined for the entire CFG G_r. This is done via two additional nodes, both mapped to the $Nop()$ action ($a_{node}(v_{entry}(G_r)) = a_{node}(v_{exit}(G_r)) = Nop()$), and then adding proper edges to connect them, respectively, with the entry and exit nodes of each production having `Program` (the start symbol) as its right-hand side. Algorithm 4 in §B.1 summarizes the contents of this section by showing the complete CFG construction algorithm from a set of productions and the control-flow graphs of the respective semantic actions.

4.4 Visiting the CFG

The control-flow graph is a representation of all possible execution paths for a role: for each parse tree that can be produced according to the grammar, the CFG contains a corresponding valid path, representing the sequence of operations performed during the tree visit. Since most practical grammars are recursive, the CFG generally admits infinitely many such paths (§4.4.8 discusses this problem and its solution in detail). The definitions in this section are set up so that they remain meaningful even though the visit algorithm presented later in this section will ultimately only explore a carefully bounded, finite subset of them.

4.4.1 Execution State Model. The simulated execution state is a tuple (v, w, S, C) such that

- $v \in V_r$ is the node currently being visited;
- $w \in V_r^*$ is the path followed to reach v;
- S is the stack of edge tags;
- $C \in \mathcal{C}$ is the stack of production contexts, where $\mathcal{C}$ is used to refer to the (infinite) set of all possible production context stacks.

To ground the definitions of this section, consider again the semantic action of Listing 5 and its CFG, shown in Figure 1 with each node labeled by a hexadecimal identifier. When `nlgcheck` begins visiting this CFG, the initial state is $(\texttt{68d651f2}, \epsilon, \epsilon, \langle c_0 \rangle)$: `68d651f2` is $v_{entry}(G)$ (mapped to $Nop()$), the path walked so far and the edge tag stack are both empty, and $C = \langle c_0 \rangle$ contains only the root production context, with m_0 initially empty.

Items of the edge tag stack $S = s_0 s_1 \dots s_n$ are edge tags ($s_i \in L$); moreover, if S is empty, we say that $\nexists pop(S)$ and $\nexists top(S)$. Instead, the production context stack $C = c_0 c_1 \dots c_n$ always contains at least one item c_0 (representing the AST root), which consists solely of an attribute mapping function $m_0 : \mathbb{N} \times I \to T$ (recall that I is the set of valid attribute identifiers, §4.2), mapping an index and an attribute name to the attribute's type: since the root has no parent to delegate to, m_0 stores both its own attributes (at index 0) and those of its children (at index ≥ 1). For $i \geq 1$, the generic c_i item represents a virtual AST node defined as $c_i = (k_i, m_i)$, a tuple where $k_i \in \mathbb{N}_{>0}$ indicates the position of the node in the child list of its parent, and $m_i : \mathbb{N}_{>0} \times I \to T$ is an attribute mapping function analogous to m_0, with the difference that m_i only maps the attributes of the node's children (at index ≥ 1): the node's own attributes (at index 0) are instead stored in its parent's mapping function m_{i-1}, indexed by the node's own position k_i.

The traditional operations offered by the stack data structure are sufficient to express most of the actions associated with a CFG node as context-stack transformations. One notable exception is the $PropagateAttrs()$ action, which needs to enumerate all the attributes within a node. We therefore extend the instruction set with a $getAll : \mathcal{C} \times \mathbb{N} \to \mathcal{P}(I \times T)$ operation defined as

$$getAll(C, i) = \begin{cases} \{(\eta, t) : ((i, \eta) \mapsto t) \in m_0\} & \text{if } C = c_0 \\ \{(\eta, t) : ((i, \eta) \mapsto t) \in m_n\} & \text{if } C = c_0 \dots c_n \text{ and } i \geq 1 \\ \{(\eta, t) : ((k_n, \eta) \mapsto t) \in m_{n-1}\} & \text{if } C = c_0 \dots c_n \text{ and } i = 0 \end{cases}$$

$getAll$ computes a set $X \subset (I \times T)$ of name-type pairs of the attributes at the i-th position of the current node. $PropagateAttrs()$ also sets the attribute name-type pairs into the current node, without replacing pre-existing mappings; this behavior is represented by the $merge : (\mathbb{N} \times I \to T) \times \mathcal{P}(I \times T) \times \mathbb{N} \to (\mathbb{N} \times I \to T)$ and $putAll : \mathcal{C} \times \mathbb{N} \times \mathcal{P}(I \times T) \to \mathcal{C}$ operations:

$$merge(m, X, i) = m \cup \{(i, \eta) \mapsto t : (\eta, t) \in (X \setminus \{(\eta', t') : ((i, \eta') \mapsto t') \in m\})\}$$

$$putAll(C, i, X) = \begin{cases} merge(m_0, X, i) & \text{if } C = c_0 \\ c_0 \dots c_{n-1}(k_n, merge(m_n, X, i)) & \text{if } C = c_0 \dots c_n \text{ and } i \geq 1 \\ c_0 \dots (k_{n-1}, merge(m_{n-1}, X, k_n))c_n & \text{if } C = c_0 \dots c_n \text{ and } i = 0 \end{cases}$$

Since the *put* only modifies at most the top of the context stack, let us introduce two more functions for brevity:

$$c_{put1}(c_{n-1}, k_n, i, \eta, t) = \begin{cases} c_{n-1} & \text{if } i \geq 1 \\ (k_{n-1}, m_{n-1} \cup \{(k_n, \eta) \mapsto t\}) & \text{if } i = 0 \end{cases}$$

$$c_{put2}(c_n, i, \eta, t) = \begin{cases} (k_n, m_n \cup \{(i, \eta) \mapsto t\}) & \text{if } i \geq 1 \\ c_n & \text{if } i = 0 \end{cases}$$

Notice that *put* (not to be confused with the *push* operation used by *EnterCtx* to push a new production context onto the stack) can be defined in terms of these two functions:

$$put(C, i, \eta, t) = \begin{cases} m_0 \cup \{(i, \eta) \mapsto t\} & \text{if } C = c_0 \\ c_0 \dots c_{put1}(c_{n-1}, k_n, i, \eta, t)c_{put2}(c_n, i, \eta, t) & \text{if } C = c_0 \dots c_n \end{cases}$$

The functions *getAll*, *merge*, and *putAll* are only relevant to *PropagateAttrs*(), which does not appear in Listing 5. The function *put*, instead, is exercised by every *WriteAttr* and *CopyAttr* node of Figure 1. For instance, node `302edb74` is mapped to $CopyAttr(0, val, 2, val)$: assuming the production context stack C already has $Op[2].val$ set to some value of type t, evaluating this node computes $put(C, 0, val, t)$, storing that same value and type under attribute *val* of *Op* itself.

4.4.2 Violations. We are now ready to define the kinds of incorrect or undesirable execution that can be detected while visiting the CFG, which we will call the set $\mathcal{V}$ of *violations*.

$MissingAttr(i, \eta)$, with $i \in \mathbb{N}$ and $\eta \in I$

This violation indicates that a semantic action accesses an attribute named η on the i-th nonterminal of its attached production that was not previously set.

$BadAttrT(i, \eta, t_r, t_p)$, with $i \in \mathbb{N}$, $\eta \in I$ and $t_r, t_p \in T$

This violation indicates that a semantic action accesses an attribute named η on the i-th nonterminal of its attached production and casts it to t_r (*required* type), which was previously set to a value of type t_p (*provided* type) not assignable to t_r ($t_p \not\leq: t_r$). While this is indeed the kind of error a static type system would normally rule out, `Neverlang` has none spanning separately compiled modules: attributes are generated as plain `Object`-typed `Java` fields, so the `Java` compiler accepts any assignment across module boundaries, and only an explicit runtime cast, inserted by `Neverlang` itself, can fail. *BadAttrT* is `nlgcheck`'s static substitute for that missing check.

$AttrTypeDynamicallyChecked(i, \eta, t)$, with $i \in \mathbb{N}$, $\eta \in I$ and $t \in T$

This violation indicates that a semantic action performs an `instanceof` (or equivalent) check against t of the type of the attribute named η on the i-th nonterminal of its attached production. This violation does not cause a runtime error but is reported as a warning to suggest that the language developer should use `Neverlang`'s type guards instead.

Consider node `5c080ef3` of Figure 1, which is mapped to $ReadAttr(1, val, Boolean)$: if the sub-expression bound to $Op[1]$ never sets an attribute named *val*, evaluating this node yields $MissingAttr(1, val)$; if $Op[1].val$ is instead set to a value of an incompatible type, say *Integer*, evaluating this node yields $BadAttrT(1, val, Boolean, Integer)$.

4.4.3 Mapping CFG Actions to State Transformations. Each of the aforementioned actions can be formally defined in terms of the transformations it performs on the state and the violations it detects: an action $a \in A$ does not manipulate or access the entire visit state, but rather just the production context stack C. We can therefore define f_{action} as a function $(C, A) \to (C, \mathcal{P}(\mathcal{V}))$ mapping a production context stack and an action to a new production context stack and a set of violations.

Here follows a list of the f_{action} with respect to the actions that can appear in the CFG:

- $f_{action}(C, WriteAttr(i, \eta, t)) = (put(C, i, \eta, t), \emptyset)$
- $f_{action}(C, ReadAttr(i, \eta, t)) = \begin{cases} (C, \{MissingAttr(i, \eta)\}) & \text{if } \nexists get(C, i, \eta) \\ (C, \{BadAttrT(i, \eta, t, get(C, i, \eta))\}) & \text{if } get(C, i, \eta) \not\leq: t \\ (C, \emptyset) & \text{otherwise} \end{cases}$
- $f_{action}(C, CopyAttr(i_d, \eta_d, i_s, \eta_s)) = \begin{cases} (C, \{MissingAttr(i_s, \eta_s)\}) & \text{if } \nexists get(C, i_s, \eta_s) \\ (put(C, i_d, \eta_d, get(C, i_s, \eta_s)), \emptyset) & \text{otherwise} \end{cases}$
- $f_{action}(C, CheckAttrType(i, \eta, t)) = (C, \{AttrTypeDynamicallyChecked(i, \eta, t)\})$
- $f_{action}(C, PropagateAttrs()) = (putAll(C, 0, getAll(C, 1)), \emptyset)$
- $f_{action}(C, EnterCtx(i)) = (push(C, (i, \emptyset)), \emptyset)$
- $f_{action}(C, LeaveCtx()) = (pop(C), \emptyset)$
- $f_{action}(C, Nop()) = (C, \emptyset)$

BeginEvalMetaAction and *EndEvalMetaAction* nodes, like $Nop()$, always leave C unchanged and never produce violations: they only affect control flow (*i.e.*, which node is visited next), which is instead governed by f_{edge} (§4.4.7).

For example, consider the path through Figure 1 corresponding to the `then` branch, assuming $Op[1].val$ was previously set to `true`: nodes `68d651f2`, `1442f788`, and `113e13f9` (respectively $Nop()$, $BeginEvalMetaAction(1)$, and $EndEvalMetaAction()$) leave C unchanged; node `5c080ef3` ($ReadAttr(1, val, Boolean)$) checks that $Op[1].val$ is set and assignable to *Boolean*; nodes `44e93c1f` and `42373389` (again $BeginEvalMetaAction(2)$ and $EndEvalMetaAction()$) leave C unchanged; and node `302edb74` ($CopyAttr(0, val, 2, val)$) finally computes $put(C, 0, val, t)$ as illustrated above.

Note that, unlike *ReadAttr*, *CopyAttr* can only produce a *MissingAttr* violation, never a *BadAttrT* one. This is not an oversight: $ReadAttr(i, \eta, t)$ carries an explicit *required* type t against which the attribute's actual type is checked, whereas $CopyAttr(i_d, \eta_d, i_s, \eta_s)$ has no such parameter. As already noted in §4.2, it copies the attribute's value *and* its type verbatim, so there is no declared type it could mismatch against. This mapping from CFG actions to context-stack transformations and detected violations can be seen as a form of *collecting semantics* in the sense of abstract interpretation [28]: each f_{action} soundly over-approximates the concrete runtime behavior of its corresponding instruction with respect to the violations of interest. The result of an f_{action} is made of two elements. We refer to the *next context* element of f_{action} with f_{aCtx}:

$$f_{aCtx}(C, a) = C' \iff \exists U : f_{action}(C, a) = (C', U)$$

We refer to the *violations* part of f_{action} with f_{aErr}:

$$f_{aErr}(C, a) = U \iff \exists C' : f_{action}(C, a) = (C', U)$$

Finally, we define $f_{pCtx} : (V_r^* \times C) \to C$ as the overall changes made to the production context stack during the traversal of a CFG path:

$$f_{pCtx}(w, C) = \begin{cases} C & \text{if } w = \epsilon \\ f_{pCtx}(w', f_{aCtx}(C, a_{node}(v))) & \text{if } w = vw' \end{cases}$$

Lemma 1. *f_{pCtx} can be evaluated piecewise on a path by composing evaluations of f_{pCtx} on parts of the path:*

$$\forall w_1, w_2 \in V_r^*, C \in \mathcal{C}, \quad f_{pCtx}(w_1 w_2, C) = f_{pCtx}(w_2, f_{pCtx}(w_1, C))$$

Proof. This can be proven by induction on w_1. The base case $w_1 = \epsilon$ is trivial, as by definition $f_{pCtx}(\epsilon, C) = C$. For the inductive step, given w_1, we assume

$$\forall w_2 \in V_r^*, C' \in \mathcal{C}, \quad f_{pCtx}(w_1 w_2, C') = f_{pCtx}(w_2, f_{pCtx}(w_1, C'))$$

and prove that

$$\forall v \in V_r, w_2 \in V_r^*, C \in \mathcal{C}, \quad f_{pCtx}(v w_1 w_2, C) = f_{pCtx}(w_2, f_{pCtx}(v w_1, C))$$

using the definition of f_{pCtx} and the inductive hypothesis with $C' = f_{aCtx}(C, a_{node}(v))$. Please refer to §C for the complete, step-by-step proof. □

4.4.4 Action Properties. All actions, except for $EnterCtx(i)$ or $LeaveCtx()$, only depend on and affect the contents of the two topmost productions of the stack. More specifically, it can depend on the attributes present at any position in the node at the top of the stack and on those on the node at position 0 of the context right under the top of the stack. We refer to this information as the *head* of the production context stack:

$$head(C) = \begin{cases} c_0 & \text{if } C = c_0 \\ m_n \cup \{(0, \eta) \mapsto t : ((k_n, \eta) \mapsto t) \in m_{n-1}\} & \text{if } C = c_0 \dots c_n \end{cases}$$

Since Figure 1 shows the CFG of a single semantic action—not yet connected to any other via $EnterCtx$/$LeaveCtx$, which only appear once semantic actions are composed into the full CFG G_r of a role (§4.3)—the production context stack C remains $\langle c_0 \rangle$ throughout its visit. Consequently, $head(C) = c_0 = m_0$ at every node of Figure 1.

Lemma 2. *The head of a production context stack only depends on its two topmost items:*

$$\forall\, C_1 = \langle c_0, \dots, c_n \rangle, C_2 = \langle c'_0, \dots, c'_n \rangle \in \mathcal{C},$$
$$c_{n-1} = c'_{n-1} \;\wedge\; c_n = c'_n \implies head(C_1) = head(C_2)$$

Proof. Trivial by definition of head. □

Lemma 3. *f_{aErr} only depends on the head of the production context stack:*

$$\forall C_1, C_2 \in \mathcal{C}, a \in A, \quad head(C_1) = head(C_2) \implies f_{aErr}(C_1, a) = f_{aErr}(C_2, a)$$

Proof. By definition of f_{action} and a_{aErr}, if a is one among $WriteAttr$, $PropagateAttrs$, $EnterCtx$, $LeaveCtx$, or Nop, then $f_{aErr}(C, a) = \emptyset \;\forall C$, trivially proving the property. Similarly, if $a = CheckAttrType(i, \eta, t)$, then $f_{err}(C, a)$ is independent of C. Finally, if $a = ReadAttr$ or $a = CopyAttr$, the contents of $f_{aErr}(C, a)$ depend on the value of a *get* operation performed on C: the value of *get* applied to a context stack $\langle c_0, \dots, c_n \rangle$ depends on either m_n or $m_{n-1}(k_n, \eta)$, both of which are equal in C_1 and C_2 under the hypothesis $head(C_1) = head(C_2)$ by definition of *head*. □

4.4.5 Balanced Action Sequences. Just like tagged edges (§4.4.7) model call/return as matching brackets on the edge tag stack, actions performed on the *production context* stack lend themselves to the same analogy, although here only two actions are relevant: $EnterCtx(i)$ (corresponding to opening a parenthesis, since it pushes a new production context) and $LeaveCtx()$ (closing the parenthesis, since it pops one). Just as in a well-parenthesized expression every closing bracket matches the nearest still-open one, every $LeaveCtx()$ in a balanced action sequence pops exactly the production context that the nearest preceding, still-unmatched $EnterCtx(i)$ had pushed. We define the *set of balanced action sequences*, $\mathcal{A}_b$, with $\mathcal{A}_b \subset A^*$ as follows:

(1) the empty sequence is balanced ($\epsilon \in \mathcal{A}_b$);
(2) given $b \in \mathcal{A}_b$ and $a \in A \setminus (\{EnterCtx(i) : i \in \mathbb{N}\} \cup \{LeaveCtx()\})$, $ab \in \mathcal{A}_b$ and $ba \in \mathcal{A}_b$;
(3) given $a_1 = EnterCtx(i)$ for any i, $a_2 = LeaveCtx()$, and $b \in \mathcal{A}_b$, $a_1 b a_2 \in \mathcal{A}_b$.

We also define the set $\mathcal{A}_{pb}$ of *prefixes* of balanced action sequences as follows:

(1) the empty sequence is a prefix of a balanced action sequence ($\epsilon \in \mathcal{A}_{pb}$);
(2) given $b \in \mathcal{A}_{pb}$ and $a \in A \setminus (\{LeaveCtx()\})$, $ab \in \mathcal{A}_{pb}$ and $ba \in \mathcal{A}_{pb}$;
(3) given $a_1 = EnterCtx(i)$ for any i, $a_2 = LeaveCtx()$, and $b \in \mathcal{A}_{pb}$, $a_1 b a_2 \in \mathcal{A}_{pb}$.

Lemma 4. $\mathcal{A}_b \subset \mathcal{A}_{pb} \subset A^*$

Proof. Trivial by definition of $\mathcal{A}_b$ and $\mathcal{A}_{pb}$. □

4.4.6 Balanced Paths. For the application of balanced action sequences in CFG paths, let us also introduce the concept of *balanced paths—i.e.*, a sequence of nodes whose associated actions form a balanced action sequence. We call W_b the set of balanced paths, with $W_b \subset V_r^*$, such that:

$$W_b = \{(v_1 \dots v_n) : \ (a_1 \dots a_n) \in \mathcal{A}_b, \text{ with } a_i = a_{node}(v_i)\ \forall i\}$$

We define the set of balanced path prefixes, W_{pb}, with $W_b \subset W_{pb} \subset V_r^*$, as

$$W_{pb} = \{(v_1 \dots v_n) : \ (a_1 \dots a_n) \in \mathcal{A}_{pb}, \text{ with } a_i = a_{node}(v_i)\ \forall i\}$$

We define a generalization, top_k, of the *top* stack operation, which allows *peeking* at the k topmost items of the stack:

$$top_k(\langle c_0, \dots, c_n \rangle) = \langle c_{n-k+1}, \dots, c_n \rangle \quad \text{with } 1 \leq k \leq n+1$$

Lemma 5. *top and top_1 are equivalent.*

Proof. Trivial by definition of top_k. □

Lemma 6. *Given an integer $k > 0$ and two production context stacks sharing the same top k items, the evaluation context stacks obtained by evaluating a balanced path prefix that does not contain CopyAttr or PropagateAttrs actions share the top k items:*

$$\forall w \in W_{pb}, \forall C_1, C_2 \in \mathcal{C}, k \in \mathbb{N}$$
$$top_k(C_1) = top_k(C_2) \implies top_k(f_{pCtx}(w, C_1)) = top_k(f_{pCtx}(w, C_2))$$

Proof. This property is proven by induction on each of the following four cases:

(1) $w = \epsilon$;
(2) $w = vw'$, with $w' \in W_{pb}$ and $a_{node}(v) = a$ with $a \in A \setminus \{LeaveCtx()\}$;
(3) $w = w'v$, with $w' \in W_{pb}$ and $a_{node}(v) = a$ with $a \in A \setminus \{LeaveCtx()\}$;
(4) $w = v_1 w' v_2$ with $w' \in W_{pb}$, $a_{node}(v_1) = a_1 = EnterCtx(i)$, and $a_{node}(v_2) = a_2 = LeaveCtx()$.

Please refer to §C for the complete proof. □

Lemma 7. *Given two production context stacks with the same head, evaluating a balanced path prefix without CopyAttr or PropagateAttrs actions produces results with the same head.*

$$\forall w \in W_{pb}, \forall C_1, C_2 \in \mathcal{C}$$
$$head(C_1) = head(C_2) \implies head(f_{pCtx}(w, C_1)) = head(f_{pCtx}(w, C_2))$$

Proof. This property is similar to Lemma 6 with $k = 2$. All considerations made on $top_3(C)\ \forall C$ also hold for the conjunction to $top_2(C)$ and $head(pop(C))$. When $head(C_1) = head(C_2)$, we have that

$$head(put(C_1, i, \eta, t)) = head(put(C_2, i, \eta, t))$$

With $C_1 = \langle c_0, \ldots, c_n \rangle$ and $C_2 = \langle c'_0, \ldots, c'_n \rangle$, that can be expanded as

$$head(\langle c_1, \ldots, c_{put1}(c_{n-1}, k_n, i, \eta, t), c_{put2}(c_n, i, \eta, t) \rangle) =$$
$$head(\langle c'_1, \ldots, c_{put1}(c'_{n-1}, k'_n, i, \eta, t), c_{put2}(c'_n, i, \eta, t) \rangle)$$

This can be proven by definition of *head*, since $m_{put2}(c_n, i, \eta, t) = m_{put2}(c'_n, i, \eta, t)$, and by considering that the mappings for k_n contained in $m_{put1}(c_{n-1}, k_n, i, \eta, t)$ are the same as those for k'_n contained in $m_{put1}(c'_{n-1}, k'_n, i, \eta, t)$, in which m_{put1} and m_{put2} are used to denote the "m-part" of c_{put1} and c_{put2}, respectively.

Then, the proof of Lemma 6 can be adapted with $k = 2$, using *head* in place of top_2, proving this property. □

Lemma 8. *Given any two production context stacks, evaluating a balanced path prefix that does not contain CopyAttr or PropagateAttrs actions, wrapped between two EnterCtx nodes, produces context stacks with the same head.*

$$\forall C_1, C_2 \in \mathcal{C}, w \in W_{pb}, \quad v_{sep} \in V_r^* : \exists i : a_{node}(v_{sep}) = EnterCtx(i),$$
$$head(f_{pCtx}(v_{sep} w v_{sep}, C_1)) = head(f_{pCtx}(v_{sep} w v_{sep}, C_2))$$

Proof. The proof follows the same structure as that of Lemma 6, according to the four possible definitions of a balanced path. Please refer to §C for the full proof. □

Lemma 9. *Given an initial production context stack $C \in \mathcal{C}$, two balanced path prefixes $w_1, w_2 \in W_{pb}$, and a production entry node v_{sep} mapped to an EnterCtx action, evaluating either $v_{sep} w_1 v_{sep} w_2 v_{sep}$ or $v_{sep} w_2 v_{sep}$ on C produces resulting context stacks with the same head.*

$$\forall C \in \mathcal{C},\ w_1, w_2 \in W_{pb}, \quad v_{sep} \in V_r^* : \exists i : a_{node}(v_{sep}) = EnterCtx(i),$$
$$head(f_{pCtx}(v_{sep} w_1 v_{sep} w_2 v_{sep}, C)) = head(f_{pCtx}(v_{sep} w_2 v_{sep}, C))$$

Proof. This property is a trivial consequence of Lemma 8. By definition of f_{pCtx}, the goal can be rewritten as

$$head(f_{pCtx}(v_{sep} w_2 v_{sep}, f_{pCtx}(v_{sep} w_1, C))) = head(f_{pCtx}(v_{sep} w_2 v_{sep}, C))$$

which is proven by applying Lemma 8 with $C_1 = f_{pCtx}(v_{sep} w_1)$, $C_2 = C$, and $w = w_2$. □

4.4.7 Following Valid Paths. In §4.3, we discussed the problem of correctly modeling control-flow transfers between different semantic actions in a CFG by representing transitions that can happen at runtime, as well as the concepts of *tagged entry and exit edges* and *valid path*.

The *edge tag stack* S, previously introduced in this section as part of the state of a CFG visit, is used to keep track, at any given point of the visit, of the tagged entry edges for which a corresponding tagged exit edge has not yet been traversed.

We define the f_{edge} function as a way to drive the algorithm through valid paths according to these concepts. It determines if an edge can be followed and updates S if needed. Let L^* be the set of all possible edge tag stacks; $f_{edge} : (L^* \times E_r) \to L^*$ is defined as

$$f_{edge}(S, e) = \begin{cases} S & \text{if } e \notin (E_{entry} \cup E_{exit}) \\ push(S, l_{edge}(e)) & \text{if } e \in E_{entry} \\ pop(S) & \text{if } e \in E_{exit} \wedge l_{edge}(e) = top(S) \end{cases}$$

Therefore, f_{edge} is not defined if following the edge would lead to an invalid path. Otherwise, f_{edge} returns the updated edge tag stack after the edge is traversed.

To illustrate tagged edges and recursion concretely, reconsider Listing 5 and Figure 1 (G), and recall that `Expression` occurs recursively on the right-hand side of the `Op` production, so a ternary

expression's branches may themselves be ternary expressions. When building the role's CFG G_r (§4.3), if $Op[2]$ happens to be bound to another ternary expression, node `44e93c1f` (mapped to $BeginEvalMetaAction(2)$) becomes $v_{beginEval}$ of a tagged entry edge to $v_{entry}(G)$ = `68d651f2`—connecting G to itself, since the callee is compiled from the same production. Let l be the tag statically assigned to this entry edge and to its corresponding tagged exit edge (`3ad4a7d6`, `42373389`).

Now consider the doubly nested expression `a ? (b ? (f ? g : h) : d) : e`: visiting its outermost `Op` node traverses the tagged entry edge (`44e93c1f`, `68d651f2`) once, pushing l onto S, to visit the middle `Op`; it is then traversed a second time to visit the innermost `Op`, at which point l occurs twice on S. A third level of nesting would traverse the same tagged entry edge a third time—which Constraint (a) forbids. This is not because a triply nested ternary cannot be validated, but because, by Theorem 1, any violation only reachable via a third traversal of (`44e93c1f`, `68d651f2`) is also reachable via an equivalent path in which it is traversed just twice.

4.4.8 Solution to the Problem of Infinite Paths. Most practical context-free grammars are recursive. A parser built from a recursive grammar can parse an infinite set of input strings; therefore, the number of parse trees that can be generated is also infinite. Even though, according to our model, different parse trees can be mapped to the same CFG visit path, the paths themselves are potentially infinite nonetheless.[10]

Thus, it is fundamental to determine a finite subset of all possible paths to render the static analysis feasible.

Naïve solutions. By keeping track of the set of already visited nodes, one can prevent visiting the same node more than once. It can be trivially shown that this approach can easily cause inconsistency, leaving the violations presented in §4.4.2 undetected even in small CFGs: take, for instance, an `if-then` statement in which an attribute is set within the body when the condition is true; then, the same attribute is accessed outside. This should lead to an error, as the attribute is not set unless the condition holds. It is possible to find similar counterexamples even when keeping track of all navigated edges instead of nodes. Thus, naïve solutions do not suffice, and a more complex one must be adopted.

Relation to classical circular-attribute handling. The problem above resembles, but is not the same as, the handling of *circular attributes* in classical AG systems, where the grammar's declared attribute dependencies form a cyclic graph. Two established strategies address that problem. *Ordered attribute grammars* [59] forbid such cycles outright: from the grammar's declared dependencies alone, a single visit sequence is computed once, statically, guaranteeing that every attribute is assigned exactly once before it is ever read, so that no repeated computation is needed at evaluation time. *Circular attribute grammars* [39], by contrast, allow genuine cycles and resolve them by repeatedly re-evaluating the cyclic attributes' equations, for each concrete input tree, until their values converge to a fixed point.

Both strategies operate on attribute *values*: an ordered AG's static ordering is a plan for computing them without redundant work, while a circular AG's fixed-point iteration is a mechanism for actually computing them despite the redundancy. The problem tackled in this section is different in kind, not merely in degree: `nlgcheck` never computes an attribute's value, only whether every read along a path is preceded by a matching write (§4.2). What may be unboundedly long or numerous here are the CFG paths themselves, purely as a consequence of the grammar—and, transitively, its compiled semantic actions—being recursive, regardless of whether any individual attribute's dependencies

[10]Two trivial examples are when a semantic action may call itself, possibly looping any number of times, or when the semantic action itself contains a loop that may *eval* another action any number of times.

are themselves circular. Consequently, neither a static evaluation ordering nor a fixed-point value iteration directly applies: there is no fixed point to converge to, since nothing is being evaluated. Constraint (a), introduced below, instead bounds how many times the *same* recursive call site may be revisited while exploring a single path—the closest analogue, in this setting, to the termination arguments used for ordered and circular AGs, even though it addresses a structurally different problem.

Complete solution. Let $W \subseteq V_r^*$ be the (potentially infinite) set of valid paths. The goal is to find a finite set $W_a \subseteq W$ such that for every node v_e, if there exist some paths from the entry node to v_e that may cause violations, then at least one such path is contained in W_a.

Let us introduce a function, f_{pErr}, that computes the set of violations detected within a path. Given a path $w \in V_r^*$ and a production context stack $C \in \mathcal{C}$, $f_{pErr} : (V_r^* \times \mathcal{C}) \to \mathcal{P}(\mathcal{V})$ is defined as follows:

$$f_{pErr}(w, C) = \begin{cases} \emptyset & \text{if } w = \epsilon \\ f_{aErr}(f_{pCtx}(w', C), a_{node}(v)) & \text{if } w = w'v \end{cases}$$

Notice that f_{pErr} *discards* any violations detected prior to reaching the last node. As we will show with the following properties and requirements, this does not affect the validity of the solution.

Lemma 10. *If w is a prefix of a balanced path, then $f_{pErr}(w, C)$ only depends on the head of C.*

$$\forall w \in W_{pb}, C_1, C_2 \in \mathcal{C}, \quad head(C_1) = head(C_2) \implies f_{pErr}(w, C_1) = f_{pErr}(w, C_2)$$

Proof. If $w = \epsilon$, the proof is trivial, as by definition, $f_{pErr}(\epsilon, C) = \emptyset \; \forall C$.
If $w = w'v$, the goal is to prove that

$$f_{aErr}(f_{pCtx}(w', C_1), a_{node}(v)) = f_{aErr}(f_{pCtx}(w', C_2), a_{node}(v))$$

For defining a prefix of a balanced path, and since $w \in W_{pb}$, we can conclude that $w' \in W_{pb}$. Therefore, by Lemma 7 and from the assumption $head(C_1) = head(C_2)$ follows that $head(f_{pCtx}(w', C_1)) = head(f_{pCtx}(w', C_2))$. Applying Lemma 3 with the context stacks resulting from $f_{pCtx}(w', C_1)$ and $f_{pCtx}(w', C_2)$ as C_1 and C_2, respectively, and $a_{node}(v)$ as a concludes the proof. □

For conciseness, let us also define $v_0 = v_{entry}(G_r)$ and $f_e(w) = f_{pErr}(w, \langle\emptyset\rangle)$. Then:

$$\forall v_e \in V_r, \quad \exists w_e = v_0 \dots v_e : f_e(w_e) \neq \emptyset \implies \exists w'_e = v_0 \dots v_e \in W_a : f_e(w'_e) \neq \emptyset \tag{1}$$

Note that a path in W_a does not *necessarily* lead to a violation.[11]

There are a few constraints that the paths must adhere to, to satisfy these requirements while keeping W_a finite:

Constraint (a): bounded recursion. A tagged entry edge is not followed if its tag is already present twice on the current edge tag stack.[12] Formally, we define a function $f_{stack} : (V_r^* \times L^*) \to L^*$ representing how traversing a path mutates the edge tag stack.

$$f_{stack}(w, S) = \begin{cases} S & \text{if } |w| \leq 1 \\ f_{stack}(v_2 w', f_{edge}(S, (v_1, v_2))) & \text{if } w = v_1 v_2 w' \end{cases}$$

Thus, Constraint (a) is expressed as:

$$W'_a = \{w \in V_r^* : \quad \nexists l_1, l_2, l_3 \in f_{stack}(w, \epsilon) : l_1 = l_2 = l_3\} \tag{2}$$

[11] The implication is not bidirectional.
[12] Notice that this condition is less strict than that of only following any given tagged entry edge at most twice in a path: if $e_1 \in E_{entry}$ is followed, and then in the same path $e_2 \in E_{exit}$ such that $l_{edge}(e_2) = l_{edge}(e_1)$ is also followed, e_1 can be followed again twice.

Intuitively, a tag being present once on the stack tests invocation, a tag being present twice tests recursion, whereas three or more occurrences do not provide additional information: this last point is not merely an empirical assumption but is formally established by Theorem 1 below, which proves that any path relying on a third occurrence of the same tag to reach a violation can always be shortened to an equivalent path in which the tag occurs only twice, without losing the violation. This is what justifies cutting off recursion at depth 2 rather than 1 (which would miss violations that only manifest across a recursive call) or some higher depth (which would be unnecessary, and would make W'_a needlessly larger). Note that Constraint (a) can only ever discard paths from the (infinite) set V_r^* of already-valid CFG paths—it never introduces a path that would not otherwise correspond to a real execution. Consequently, restricting the search to W'_a can only cost completeness (a real violation being missed because every path exhibiting it was excluded), never soundness: whenever a violation is reported along some $w \in W'_a$, w is by construction a genuine valid path of the CFG, so the violation it exposes is a real one, not an artifact of the depth cutoff. Theorem 1 then rules out the completeness risk as well, by showing that no violation is exclusively reachable through a third or later occurrence of a tag.

Constraint (b): no redundant revisits. A node is not visited again if it was previously visited with the same production context stack and edge tag stack as the current ones. Formally:

$$W_a = \{w = v_0 \dots v_n \in W'_a : \quad \forall w' = v'_0 \dots v'_n \in W'_a \text{ such that } w' \neq w \wedge v'_n = v_n, \\ f_{pCtx}(w, \langle\emptyset\rangle) \neq f_{pCtx}(w', \langle\emptyset\rangle) \vee f_{stack}(w, \epsilon) \neq f_{stack}(w', \epsilon)\} \tag{3}$$

It can be observed that $W_a \subseteq W'_a$.

We now prove two lemmas demonstrating that under the assumption that no *CopyAttr* or *PropagateAttrs* nodes are present in the graph (which is a precondition to several properties proven thus far), visiting all the paths contained in the set W_a determined by the constraints above is sufficient to find a violation within a path, if any exists.

A limitation of this approach is that we cannot prove that the same property holds when *CopyAttr* or *PropagateAttrs* actions are used. However, we deemed this limitation acceptable for the following reasons.

(1) While lacking a formal proof, experimentation (see §5) demonstrated that even when such actions are used, the software is able to find most violations anyway in practice.
(2) All *PropagateAttrs* actions can always be replaced by copying relevant attributes from child notes to parent nodes explicitly, instead of relying on this quality-of-life feature. Such a feature is not specific to `Neverlang`: other AG systems provide comparable means of automatically propagating attributes without a dedicated equation for each one, such as the `CHAINS` construct of `Eli` [46]. Similarly, *CopyAttrs* actions can also be replaced by other, more explicit, actions: semantic instructions of the form `$0.attrA = $1.attrB` can be replaced with `$0.attrA = (T) $1.attrB`, where T is the expected type of the attribute, thus modifying the CFG accordingly, substituting the single $CopyAttr(0, attrA, 1, attrB)$ node with the two nodes $ReadAttr(1, attrB, T)$ and $WriteAttr(0, attrA, T)$ in succession. In other words, the problem is solved by rendering the type of the attribute explicit. By taking these precautions, the approach is formally sound and can always find any violations.

Theorem 1. *W'_a, restricted by Constraint (a) alone as defined by (2), satisfies requirement (1). In fact, the proof establishes the stronger property that w'_e can always be chosen such that $f_e(w'_e) = f_e(w_e)$, i.e., w'_e detects exactly the same violations as w_e, not merely some violation.*

Proof. Let us consider a generic node $v_e \in V_r$; we must prove that if paths from the entry node to v_e with a violation exist, then at least one of them is contained within W'_a, so that it satisfies (2).

So, let's assume that

$$\exists w_e = v_0 \dots v_e : f_e(w_e) \neq \emptyset \tag{4}$$

and proceed by proving that

$$\exists w'_e = v_0 \dots v_e : \quad f_e(w'_e) \neq \emptyset \ \wedge \ w'_e \in W'_a$$

which, due to (2), can be rewritten as

$$\exists w'_e = v_0 \dots v_e : \quad f_e(w'_e) \neq \emptyset \ \wedge \ \nexists l_1, l_2, l_3 \in f_{stack}(w'_e, \epsilon) : l_1 = l_2 = l_3$$

Proceeding by contradiction, assume the goal is false. Without loss of generality, we consider that evaluating *any* path from the entry node v_0 to v_e that leads to the detection of a violation at v_e requires pushing the same edge tag on the stack at least three times. Then:

$$\forall w'_e = v_0 \dots v_e : \quad f_e(w'_e) = \emptyset \ \vee \ \exists l_1, l_2, l_3 \in f_{stack}(w'_e, \epsilon) : l_1 = l_2 = l_3 \tag{5}$$

With $w'_e = w_e$ and due to (4)

$$\exists l_1, l_2, l_3 \in f_{stack}(w_e, \epsilon) : l_1 = l_2 = l_3$$

or, in words, the edge tag stack necessarily contains the same tag l such that $l = l_1 = l_2 = l_3$ at least three times after traversing w_e.

We show the contradiction by proving that if l is present on the final edge tag stack exactly three or more times, then part of the path can be removed.

By construction of the CFG and by definition of f_{edge} and f_{stack}, condition (5) occurs when an entry edge $(v_1, v_2) \in E_{entry}$, such that $l_{edge}((v_1, v_2)) = l$, is traversed exactly three times in w_e without the corresponding exit edge $(v_3, v_4) \in E_{exit}$ such that $l_{edge}((v_3, v_4)) = l$ ever being traversed. Therefore, the (possibly empty) w_1, w_2, w_3, w_4 paths exist, such that

$$w_e = v_0 w_1 v_1 v_2 w_2 v_1 v_2 w_3 v_1 v_2 w_4 v_e$$

and such that none between w_2, w_3, and w_4 contain the vertex pair $v_3 v_4$. Recall that only valid paths (as by definition provided in §4.3) are considered; thus, w_e is a valid path. It follows that w_2, w_3, and w_4 must also be valid paths, *i.e.*, they cannot contain exit edges without a corresponding entry edge. Upon traversing each of them, the top of the edge tag stack is l; therefore, if an exit edge had to be followed, its tag would also need to be l, but this would mean that the edge is (v_3, v_4), which is not allowed under the current hypothesis.

It is true by CFG construction that each entry edge always starts from a node mapped to an *EnterCtx* action, and also that each exit edge always points to a node mapped to a *LeaveCtx* action. The inverse is also true, *i.e.*, all outgoing edges from an *EnterCtx* node are entry edges, and all incoming edges to a *LeaveCtx* node are exit edges.

We can therefore conclude that neither w_2, w_3, nor w_4 can contain *LeaveCtx* nodes unless they also contain a matching *EnterCtx* node, as by definition of *balanced path prefix* given in §4.4.5; therefore, $w_2, w_3, w_4 \in W_{pb}$.

We expand $f_e(w_e)$ as follows:

$$\begin{aligned}
&f_e(w_e) = && \text{(by definition of } f_e\text{)} \\
&= f_{pErr}(w_e, \langle\emptyset\rangle) = && \text{(by definition of } f_{pErr}\text{)} \\
&= f_{pErr}(v_0 w_1 v_1 v_2 w_2 v_1 v_2 w_3 v_1 v_2 w_4 v_e, \langle\emptyset\rangle) = && \text{(by Lemma 1)} \\
&= f_{pErr}(v_2 w_4 v_e, f_{pCtx}(v_0 w_1 v_1 v_2 w_2 v_1 v_2 w_3 v_1, \langle\emptyset\rangle)) = && \text{(by Lemma 1)} \\
&= f_{pErr}(v_2 w_4 v_e, f_{pCtx}(v_1 v_2 w_2 v_1 v_2 w_3 v_1, f_{pCtx}(v_0 w_1, \langle\emptyset\rangle)))
\end{aligned}$$

Recall that (v_1, v_2) is an entry edge and thus:

(1) v_1 is mapped to an *EnterCtx* action and is therefore compatible with the requirements v_{sep} in Lemma 9;
(2) v_2 cannot be a *LeaveCtx* action—as it is the entry node of a semantic action—and therefore $v_2w_2, v_2w_3 \in W_{pb}$.

We can then apply Lemma 9 with $C = f_{pCtx}(v_0w_1, \langle\emptyset\rangle)$, $v_{sep} = v_1$, $w_1 = v_2w_2$, and $w_2 = v_2w_3$ to obtain

$$head(f_{pCtx}(v_1v_2w_2v_1v_2w_3v_1, f_{pCtx}(v_0w_1, \langle\emptyset\rangle))) = head(f_{pCtx}(v_1v_2w_3v_1, f_{pCtx}(v_0w_1, \langle\emptyset\rangle)))$$

Since v_2 is not a *LeaveCtx* action, we know also that $v_2w_4 \in W_{pb}$, whereas from the hypothesis $f_e(w_e) \neq \emptyset$, we know that $a_{node}(v_e) \neq LeaveCtx()$, as $f_{aErr}(C, LeaveCtx()) = \emptyset\ \forall C$; thus, $v_2w_4v_e \in W_{pb}$. We can then apply Lemma 10 to obtain

$$f_e(w_e) = f_{pErr}(v_2w_4v_e, f_{pCtx}(v_1v_2w_3v_1, f_{pCtx}(v_0w_1, \langle\emptyset\rangle)))$$

from which, by Lemma 1 and definition of f_{pErr}, follows that

$$f_e(w_e) = f_{pErr}(v_2w_4v_e, f_{pCtx}(v_0w_1v_1v_2w_3v_1, \langle\emptyset\rangle)) = f_{pErr}(v_0w_1v_1v_2w_3v_1v_2w_4v_e, \langle\emptyset\rangle)$$

Let $w'_e \coloneqq v_0w_1v_1v_2w_3v_1v_2w_4v_e$ for brevity: by definition of f_e, the above chain of equalities states exactly that $f_e(w_e) = f_e(w'_e)$—not merely that both are non-empty, but that w'_e detects the same set of violations as w_e. In particular, since $f_e(w_e) \neq \emptyset$ by hypothesis (4), it follows that $f_e(w'_e) \neq \emptyset$ as well, and w'_e only contains the edge (v_1, v_2) twice; therefore, after traversing w'_e, the tag l is only present on the edge tag stack twice. This conclusion contradicts the hypothesis of the *reductio ad absurdum* (5), proving that a generic path leading to the detection of a violation can be reduced to a path, satisfying the requirement for inclusion in W'_a as specified by (2), that detects the very same violations. □

The reduction argument underlying Theorem 1—that a third traversal of the same recursive call site is always redundant, since it can be collapsed onto an equivalent path traversing it only twice—is conceptually related to recent results on context-sensitive program analysis of service-oriented systems [95], where compositions of workflow nets are similarly analyzed by bounding the recursive unfolding of net compositions to a small, fixed depth without loss of analysis precision. While the two settings differ (workflow net composition versus attribute grammar composition), both rely on the same underlying principle: once a recursive structure has been unfolded enough times to expose its repeating pattern, further unfolding cannot reveal any additional behavior that a shorter unfolding could not already exhibit.

Theorem 2. *With the additional restriction of Constraint (b), specified by* (3)*, W_a still satisfies requirement* (1).

Intuitively: if two distinct paths w, w' both reach the same node v having produced the same production context stack and the same edge tag stack (*i.e.*, $f_{pCtx}(w, \langle\emptyset\rangle) = f_{pCtx}(w', \langle\emptyset\rangle)$ and $f_{stack}(w, \epsilon) = f_{stack}(w', \epsilon)$), then, from v onward, any continuation of the visit behaves identically regardless of which of the two paths was taken to reach it. Constraint (b) exploits exactly this: it is therefore safe to keep only one of w, w' in W_a and discard the other, without missing any violation that the discarded path alone would have revealed.

Proof. The goal is to prove that, given a path $w_e \in W'_a$ from the entry node v_0 to a node v_e with a detected violation ($f_e(w_e) \neq \emptyset$), then a $w'_e \in W_a$ such that $f_e(w'_e) \neq \emptyset$ also exists.

The proof is trivial when considering that the requirement for inclusion of paths in W_a (3) can be formulated as follows: given a set of paths that all share the same results of evaluating f_{pCtx} and

f_{stack}, *i.e.*, a set W_x such that

$$\exists C \in \mathcal{C}, S \in L^*, \quad \forall w_x \in W_x \;\; f_{pCtx}(w_x, \langle \emptyset \rangle) = C \;\wedge\; f_{stack}(w_x, \epsilon) = S$$

then exactly one of them is included in W_a.

In fact, a direct consequence of such a definition is that all the paths $w_x \in W_x$ also share the same value of $f_e(w_x)$, by definition of f_e. Therefore, if $\exists w_e \in W_x$ such that $f_e(w_e) \neq \emptyset$, then regardless of which of the other items is picked from W_x to fill the role of w'_e, it satisfies $f_e(w'_e) = f_e(w_x) \neq \emptyset$. □

4.4.9 Visit Algorithm. We can finally define a theoretically sound algorithm that traverses the CFG according to the properties defined thus far to detect any of the relevant violations that may occur when composing separately compiled language fragments.

We employ the previously defined functions f_{action} and f_{edge}, respectively, to evolve the state and to drive the visit through valid paths only. The implementation keeps a set of states to visit as storage for paths that need to be explored further, and one of all states that have been visited to enforce path condition (3). Condition (2) is instead enforced by inspecting the edge tag stack before following an edge. Recall that f_{edge} is a partial function (§4.4.7): the condition $\exists S'$ in Algorithm 1 checks that $f_{edge}(S, (v, v'))$ is defined, *i.e.*, that following the edge (v, v') does not lead to an invalid path. The worklist Φ is deliberately modeled as a set, and Algorithm 1 picks an arbitrary element of it at each iteration, rather than fixing a specific traversal order (*e.g.*, breadth-first or depth-first): Theorems 1 and 2 and the soundness of the overall visit hold regardless of which state is picked at each step, thus no particular order is required for correctness, and none is imposed by the formal model. The actual implementation resolves this choice concretely by keeping Φ as a list and always continuing the visit from the states most recently discovered, *i.e.*, a depth-first traversal. This is simply the simplest instantiation of the formal model's "pick any" step, not a requirement.

4.5 Optimization

While sound, Algorithm 1 needs performance that allows static analysis to be performed as part of the language development process with ease. Therefore, significant effort went toward reducing the time complexity of the visit algorithm, with the goal of making the execution of `nlgcheck` adequate to be used with non-trivial languages. We will not discuss such optimizations in detail, so please refer to §5 for a performance evaluation. This section is limited to providing a brief overview of both optimizations and their rationale. Both optimizations rely, in different ways, on avoiding redundant work: §4.5.1 avoids constructing duplicate CFG fragments in the first place, while §4.5.2 caches, per nonterminal, which combinations of production-context-stack heads have already been explored, so that a semantic action already analyzed for a given head need not be revisited for the same head again. Please refer to §B.2 for the optimized visit algorithm.

4.5.1 Graph Deduplication. The execution time of Algorithm 1 largely depends on the size of the CFG. A reasonable optimization is therefore to reduce the size of the graph whenever possible, which in turn depends on the number of semantic actions in the language. The optimization leverages a few key empirical observations.

- Based on our own experience developing languages with `Neverlang` (Table 2), it is not uncommon for a language grammar to contain productions that differ from each other only in the terminal keyword used in their concrete syntax to disambiguate which alternative was parsed—regardless of whether that terminal itself survives as an AST node—and are therefore syntactically equivalent in this analysis context. Consider, for example, a grammar with two distinct productions for operators sharing the same precedence, such as `AddExpr: Expr ⇐ Expr "+" Term` and `SubExpr: Expr ⇐ Expr "-" Term`, which share the same nonterminals

Algorithm 1 Visit of a role's CFG to find violations

$\Phi := \{(v_{entry}(G_r), v_{entry}(G_r), \epsilon, \langle\emptyset\rangle)\}$ ▷ Set of states to visit
$\Gamma := \emptyset$ ▷ Set of visited states
$U := \emptyset$ ▷ Found violations
while $\Phi \neq \emptyset$ **do**
  pick any $(v, w, S, C) \in \Phi$
  $\Phi := \Phi \setminus \{(v, w, S, C)\}$
  if $(v, C, S) \notin \Gamma$ **then** ▷ Path condition (3)
    $\Gamma := \Gamma \cup (v, C, S)$
    $(C', U') := f_{action}(C, a_{node}(v))$
    $U := U \cup U'$
    for all $(v, v') \in E_r$ **do**
      $S' := f_{edge}(S, (v, v'))$
      if $\exists S'$ **and** $\nexists l_1, l_2, l_3 \in S : l_1 = l_2 = l_3$ **then** ▷ Path validity and (2)
        $\Phi := \Phi \cup \{(v', wv', S', C')\}$
      end if
    end for
  end if
end while
if $U = \emptyset$ **then**
  verification succeeded
else
  violation(s) have been detected and collected in U
end if

and are disambiguated in the concrete syntax by the `+` and `-` keywords, respectively. The same can be said for other binary or unary operators.

- It is also common for semantic actions associated with such productions to have very similar implementations. With respect to the prior example, the implementations of `AddExpr` and `SubExpr` are extremely similar for all intents and purposes, differing only in the arithmetic operator used to implement the expression. Most notably, such *similar* semantic actions might as well be *identical* from `nlgcheck`'s standpoint: since the CFG only tracks attribute reads, writes, and control flow, the arithmetic operator itself is irrelevant to the analysis, regardless of whether the AST retains the disambiguating terminal as a node.
- Modularity is one of the strongest points of language workbenches, and thus language developers are encouraged to keep semantic actions as small as possible, with the goal of improving their reusability. By adhering to this guideline, semantic actions can be as small as a single or a few lines of code, without any control flow. In this case, the CFGs simply consist of a chain of nodes, each of which has exactly one entry node and one exit node.

In summary, the actual output of converting semantic actions into CFG fragments may contain several duplicates. Therefore, the CFG building algorithm can be optimized by discarding any CFG of these duplicates. Detecting all such duplicates would require deciding graph isomorphism, a problem for which no polynomial-time algorithm is known in general. We sidestep it with a conservative check that exploits the observations above. Two productions are considered duplicates only if they have the same left- and right-hand-side nonterminals and, in addition, either their semantic actions are the same compiled action, or the CFGs of their semantic actions are chains of nodes, as in the third observation, mapped to the same sequence of actions. The latter comparison

takes time linear in the length of the chains. The check may miss some duplicates, such as two equivalent fragments with internal branching compiled from different semantic actions, but it never merges two fragments that differ for the analysis.

4.5.2 Path Deduplication. Consider a generic nonterminal symbol A of the grammar. Every time the semantics of a production having A in its right-hand side—*i.e.*, with the form $B \leftarrow \ldots A \ldots$—is analyzed, the semantics of all the productions with A on the left-hand side must also be analyzed.

The language potentially contains several such productions. We empirically observed that it is frequent for the *head* of the production context stack to be the same when entering one of the semantic actions among the set of those associated with productions generating the same nonterminal. There are two possible reasons for this fact.

(1) We found it to be more common to pass attributes from child to parent in the AST than from parent to child; therefore, when descending into the AST, when encountering $EnterCtx$ nodes in a CFG path, the *head* of the context stack often contains no attributes.
(2) In the cases in which attributes are passed from parent to child, we can expect the semantic actions associated with the child nodes to actually read these attributes; therefore, for all the paths without a violation[13] the *head* of the context when entering a semantic action must contain all the attributes that the semantic action will read.

The idea for this optimization leverages these observations to review the structure of the CFG: for each nonterminal A, we add an *entry node* and an *exit node* that are shared among all the semantic actions attached to productions of the form $A \leftarrow \ldots$. Such nodes serve as *anchor points* to determine when to access and when to update a cache storing all the production context stacks. By caching all the possible heads of the production context stack at the corresponding nonterminal exit node for each possible head of the production context stack at each nonterminal entry node, the algorithm can *jump* from the nonterminal entry node to the exit node. The goal is to avoid repeating the whole visit when a particular node is reached more than once with the same context stack.

4.6 Acting on Reported Violations

Our analysis is meant to be run by two kinds of users. The authors of a language component have its source code and can run `nlgcheck` on test compositions before distributing the component, as with the stand-alone test languages that accompany the sublanguages of *numericUnitExpr* (§5.1). For them, a reported violation is as actionable as a failing well-definedness check in systems such as Silver (§6). The integrator who composes precompiled components into a language, instead, only has their binaries, and cannot fix a reported violation by editing the components themselves.

For such an integrator, a violation is not a bare yes/no answer. For each violation, `nlgcheck` reports the source location of the offending attribute access, the missing or ill-typed attribute, the attributes that are actually available at that point together with their types, and a witness: the chain of productions whose evaluation leads to the access. This is usually enough to tell a naming mismatch between two components, such as one component writing `value` where another reads `val`, from a contribution that no available component provides.

Both cases can be addressed at composition time, without source code, since `Neverlang` composes languages from separately compiled modules through slices (§3.1), which are themselves small, separately compiled units. A naming mismatch can be resolved with an *attribute mapping*: a slice that imports the role of a module can rename, for that module only, the attributes accessed by its semantic actions. For instance, the slice statement

```
module M with role evaluation mapping attributes { $1.value => val }
```

[13] As is commonly done in error theory, we expect the absence of errors to be more likely than their presence.

makes every access to the attribute `value` of the first right-hand-side nonterminal, within the semantic actions of `M`, refer to `val` instead. `nlgcheck` applies the same renaming when building the CFGs of these semantic actions, so the composition obtained with the new slice can be checked again. A missing contribution can be addressed by replacing, in a new slice, the module that provides the offending role for that syntax with a module written by the integrator, which is feasible since semantic actions tend to be small (§4.5.1). Otherwise, the integrator can select a different version of a component, or report the problem to its authors.

The situation is comparable to a linker reporting an unresolved symbol while combining binary libraries. The report does not fix the problem, but it moves its discovery from the runtime of the compiler, possibly at the site of the end user, to the moment of composition, and it points to where the fix is needed.

5 Evaluation

We performed a quantitative evaluation to test the applicability of `nlgcheck` beyond its theoretical soundness. This evaluation is guided by the following research questions:

RQ1 How effective is `nlgcheck` at detecting undefined-attribute errors introduced by separately compiling and composing language components?

RQ2 What is the performance overhead of `nlgcheck`, and how do the optimizations of §4.5 affect it?

To answer RQ1, we opted for an evaluation of the results using source code mutations [34, 53] on pre-existing `Neverlang` languages, generated by a companion tool, *nlgmutator*, which is not part of `nlgcheck` itself: it instruments the `Neverlang` compiler to systematically alter attribute names within semantic actions, producing mutated `Neverlang` projects that are then independently checked with `nlgcheck`. Traditionally, the goal of mutation testing is to assess the quality of test suites. It therefore fits our requirement of systematically introducing structural errors within a language to verify whether `nlgcheck` is capable of detecting them. Moreover, mutation testing was part of a prior study with `Neverlang` [22] and was therefore deemed an adequate choice, especially because the reflection *application programming interface* (API) of `Neverlang` allows developers to easily mimic mutation operators without encompassing source code recompilation: in mutation testing, a mutation operator is a function that takes the original program and produces the mutated one.

Concerning the errors that can happen when composing language fragments, a mutation operator that is extremely easy to implement and that can mimic most real-world programming errors is the random change of one or more characters within the identifier of an attribute. This is a simple yet relevant example because, due to separate compilation, `Neverlang` cannot detect such an error statically—not knowing in advance if a typo in an attribute name was indeed an error or simply an intended design choice foreseeing more language fragments that may be implemented and compiled in the future. We are aware that this operator only introduces naming errors, and not, *e.g.*, errors in the type hierarchy such as the assignment of an incompatible-but-similarly-named type. §5.3 discusses this limitation further. The software and data used to perform this experiment are available at https://zenodo.org/records/21795001.

Experimental setup. To build the CFG required by §4.2, `nlgcheck` uses `Soot` to extract a preliminary CFG (in `Soot`'s own *Jimple* intermediate representation, §3.2) from the compiled bytecode of each semantic action. This preliminary CFG is precise—it faithfully represents the compiled method's control flow—but is expressed in terms of `Jimple` instructions rather than the actions of §4.2, so a translation step then maps it onto a CFG containing only those actions. We implemented `nlgcheck` in `Scala`, translating the formal model of §4 directly: each action of

Algorithm 2 Mutation-testing experiment performed on each evaluated language $\mathcal{L}$

```
compile the .nl sources of L into Java code, and the Java code into bytecode
if a compilation step fails or nlgcheck reports a violation on the compiled classes then
    exclude L from the experiment                                   ▷ Control run
else
    instrument the Neverlang compiler with the mutation operator
    for all attribute access expressions x of L, in textual order within each production do
        x' := x, with a random character of its identifier replaced by a different random letter
        regenerate the Java code from the unmodified .nl sources, emitting x' in place of x
        compile the mutated Java code and record whether compilation fails
        run nlgcheck on the mutated classes and record whether it reports a violation
    end for
end if
```

§4.2 (*e.g.*, *WriteAttr*) is an immutable case class implementing a common `Action` interface, whose `visit` method is a pure function computing f_{action} and returning an updated, equally immutable production context, mirroring the *put*/*get* functions of §4.4. The visit algorithm of Algorithm 1 itself is a tail-recursive function over an explicit worklist, threading the visited-states set and the collected violations as immutable function arguments rather than as mutable fields. Mutable state is used only where the surrounding framework leaves no alternative: *nlgmutator*, for instance, is driven by the `Neverlang` compiler's own translator-plugin architecture, which repeatedly invokes a callback to translate each production's semantic action. Since this callback is invoked by code outside our control, tracking which production is currently being mutated and how far into it the last mutation reached requires state that persists across those calls, and is therefore kept in a small amount of module-level mutable state, isolated from the otherwise immutable analysis core. Algorithm 2 summarizes the experiment performed on each evaluated language. The unmutated language is first compiled and checked with `nlgcheck` as a control. If either step reports a problem, the language is excluded, since any subsequent finding would then be unrelated to the mutations being tested. Mutations are then introduced one at a time by instrumenting the `Neverlang` compiler with the mutation operator. They are therefore introduced while generating `Java` code from the unmodified `.nl` sources, not by editing the `.nl` specifications themselves, and each mutant contains exactly one mutation. Attribute access expressions are selected systematically, in the textual order in which they appear within their production, until each of them has been mutated exactly once. This exhausts the space of single-attribute-name mutations for the language, so the number of generated mutants equals the total number of attribute access expressions across all the semantic actions of the language. The randomness in each mutation is confined to *how* the selected identifier is corrupted (a uniformly random character position is replaced with a uniformly random different letter), not to *which* attribute access is selected next. For each mutant, we record whether it already fails to compile to `Java` bytecode, *i.e.*, whether the baseline toolchain catches it without `nlgcheck`, and whether `nlgcheck` reports a violation.

Data Setup. At the time of writing, a number of `Neverlang`-based languages exist that can be used to evaluate `nlgcheck`. Among these, we chose a subset comprised of simple, reusable sublanguages (*expr* and *numericUnitExpr*), domain-specific languages (*stateMachines* and *logLang*), and a general-purpose programming language (*neverlangJS*). Note that *neverlangJS* is an interpreter, and follows the convention, typical of `JavaScript` interpreters, that every `Expression` and `Statement` yields a

Language	Modules	LOC
expr	9	73
numericUnitExpr	46	474
stateMachines	12	181
logLang	25	279
neverlangJS	130	4616

Table 1. Characterization of the dataset used for the mutation-testing experiment: number of `Neverlang` modules (`.nl` files) and lines of `Neverlang` code per language.

Language	Generated Mutants	Detected w/o `nlgcheck`	Detected with `nlgcheck`
expr	14	0 (0%)	14 (100%)
numericUnitExpr	77	14 (18%)	55 (71%)
stateMachines	19	0 (0%)	5 (26%)
logLang	25	0 (0%)	22 (88%)
neverlangJS	596	0 (0%)	441 (74%)
Total	731	14 (2%)	537 (73%)

Table 2. Results of running *nlgmutator* on various preexisting `Neverlang`-based projects, showing how many mutants are generated, how many are detected in the regular compilation process, and how many are detected by `nlgcheck`. Percentages are rounded to the nearest integer.

result of a common `Value` type and shares a common runtime environment. This convention is representative of interpreters in general, rather than being specific to *neverlangJS*.

Each of the chosen types of languages has a rationale.

- Sublanguages are arguably the most fit to be analyzed with `nlgcheck`, as they are intended to be reused across several languages and are therefore pre-compiled separately from the host language.
- Domain-specific languages are the main application scenario of language workbenches in general. They can also be embedded in other languages, while using syntactic constructs usually different from those of general-purpose languages, which may lead to errors.
- A full-fledged general-purpose language represents a large-scale project, often developed by several developers concurrently, each possibly adopting different choices with respect to the attribute grammar. As a result, keeping track of the entire project without relaxing the separate compilation constraint may prove challenging.

Table 1 characterizes the scale of each language in terms of the number of `Neverlang` modules and lines of `Neverlang` code it is composed of. Table 3 (§5.2) additionally reports, for each language, the number of grammar productions and CFG nodes involved in the analysis.

Hardware setup. All tests were conducted on an Intel Core i7-4720HQ machine with a maximum clock speed of 3.6GHz and 16GB of RAM. The software consists of the GNU/Linux 6.1.12 (x86_64) operating system and OpenJDK 17.0.6. Please note that a different setup may yield different results.

5.1 Error Detection

Results. Table 2 summarizes the results of the experiment. The total number and the percentage of mutants that cause the compilation process to fail are then compared to those of mutants detected by `nlgcheck`.

In the vast majority of cases, mutants survive the compilation process fully undetected, with one notable exception, *i.e.*, when an attribute appears both within the semantic action body and the type guard associated with that action. In such cases, the `Neverlang` compiler injects type casts within the generated `Java` code, and therefore, if the attribute is mutated while the guard is not, the cast is no longer injected correctly, possibly leading to `Java` compilation errors.[14] As reported in Table 2, such errors amount to only 18% of errors in the *numericUnitExpr* code base and only 2% overall (14 out of 731 across all experiments). In contrast, the total number of mutants detected by `nlgcheck` is 537 out of 731, thus about 73%.

Table 2 does not repeat the execution time of `nlgcheck` on each language, which is reported in Table 3 (§5.2) together with that of the unoptimized and partially optimized variants of the tool. It also omits a compile-only baseline without `nlgcheck`, since that overhead is orthogonal to the questions this experiment answers. Finally, unlike Table 3, it does not include *neverlangExpressions*. The control run of Algorithm 2 already reports violations on the unmutated *neverlangExpressions*, so the protocol excludes it: any violation reported on its mutants could not be attributed to the mutation. Moreover, a single run of `nlgcheck` on this language takes minutes (Table 3), which would make a campaign with one run per mutant considerably more expensive than for the other languages. Extending the campaign to a corrected version of *neverlangExpressions* is left as future work.

Discussion. Because the `Neverlang` compiler does not perform any verification of attribute name matching across multiple modules, it is apparent that it alone does not suffice to effectively capture errors in the definition of grammar attributes. Our tool represents a significant improvement when compared to this baseline. However, one may wonder about the remaining 27% of mutants that are still undetected even when using `nlgcheck`, especially given the theoretical soundness of the approach. There are a few considerations to be made in this regard.

Read without write. First, while the mutation operator has the goal of simulating programming errors that would result in a crash at runtime, not all mutations lead to errors. Consider an attribute that is written but never read: this may indicate either a design flaw or the implementation of a sublanguage that exposes attributes to be used by modules from other sublanguages.

Unnecessary copy. Another source of undetected mutants occurs when semantic actions associated with a production of the form `A` ← `B` copy an attribute from their only AST child node: as previously mentioned, attribute access to a node having exactly one child triggers a quality-of-life feature so that the AST chains are navigated from parent to child when an attribute is not found within the parent. In this particular case, copying the attribute with an incorrect name may not result in a runtime error. After performing the experiment, manual inspection of `Neverlang` code revealed such cases of unnecessary copy operations to be not uncommon, especially in the *neverlangJS* code base.

Dead code. Mutants may remain undetected because the mutated code can never be reached within the CFG; `nlgcheck` does not currently detect dead code as a violation in its own right, so this is presently only a source of missed detections rather than a planned feature. We found such

[14] Recall that in the `Java` code generated by `Neverlang`, attributes are simply assumed to be `Objects` unless stated otherwise.

an example in the *numericUnitExpr* code base, which is articulated in several sublanguages, each implementing one concern of the language, such as integer arithmetic, floating-point arithmetic, unit checking, and so on. Each of these libraries also includes portions of so-called *glue code* needed to run them in stand-alone mode—mainly for testing purposes, such as a language only implementing binary operations between integer numbers. Such pieces of code consist of a definition of the `Program` nonterminal axiom, associated with a semantic action. Since mutation operators mutate all modules, they eventually also mutate semantic actions that are not actually used in the final language. Pieces of dead code were discovered in *neverlangJS* as well.

Non-idiomatic attribute access. Under the hood, `Neverlang` can simply be seen as a `Java` API for the development of programming languages. Therefore, developers can either approach `Neverlang` development in an *idiomatic* way, by writing semantic actions in `Neverlang` source files, or in a *non-idiomatic* way, by invoking `Java` methods and interacting with the AST through the `Neverlang` runtime API, instead of using the `Neverlang` syntax. Currently, there is no support in `nlgcheck` to translate arbitrary `Java` code into a CFG compatible with the requirements presented in §4. Therefore, any *non-idiomatic* attribute access remains undetected even if the corresponding attribute was mutated upon it being written in a different semantic action. We did not perform a systematic mining of public `Neverlang` repositories to estimate how common non-idiomatic access is in practice; this is left as future work. A mechanism akin to `JastAdd`'s explicit attribute declarations—requiring non-idiomatic `Java` code to access an attribute only through a declared, module-level interface, rather than directly through the runtime API—could in principle close this gap. Such declarations would give `nlgcheck` the read/write information it currently lacks for non-idiomatic accesses, without requiring it to parse arbitrary `Java` code. `Neverlang` does not presently offer such a mechanism, so this remains a promising direction for future work rather than a solution available today.

Relation to dependency taxonomy. Revisiting the RAW/WAR/WAW hazard taxonomy of §2.2.3: the *non-idiomatic attribute access* category corresponds to a genuine, undetectable RAW hazard (a read using a value produced by a write, where the write is outside `nlgcheck`'s visibility); *read without write* and *dead code* correspond to writes with no corresponding read on any reachable path, i.e., the absence of a hazard rather than an undetected one; *unnecessary copy* corresponds to a (potential) WAW/output dependency between the copy and a same-named write elsewhere, which happens not to manifest as an observable error in this specific case.

Why *stateMachines* detects fewer mutants. *stateMachines* has the lowest detection ratio among the evaluated languages (26%, Table 2). We do not have a per-mutant breakdown to attribute an exact share of this to each of the four categories above, but re-inspecting *stateMachines*' grammar surfaces a concrete, language-specific contributor to the *read without write* category: the `StateBlock` nonterminal has an empty alternative (`StateBlock ⇐ ""`, selected whenever a state declares no outgoing events), and the enclosing `StateDeclaration` action always writes a `state` attribute onto its `StateBlock` child regardless of which alternative is parsed, yet only the non-empty alternative's semantic action ever reads it back. Since states with no outgoing events are common in typical state-machine specifications, this single write-without-read pattern alone is likely to account for a disproportionate share of *stateMachines*' undetected mutants compared to the other, less alternative-heavy languages in Table 2.

Summary. We summarized the main reasons for which mutants may be undetected by `nlgcheck`. For the *read without write, unnecessary copy,* and *dead code* mutants, reporting them as errors should

Language	Unoptimized Prods (Nodes)	Time	Opt. 1 only Prods (Nodes)	Reduction	Time	Opts. 1 & 2 Time
expr	10 (61)	✗	8 (45)	20% (26%)	3s	3s
numericUnitExpr	14 (159)	✗	13 (157)	7% (1%)	✗	7s
stateMachines	14 (65)	3s	12 (59)	14% (9%)	3s	3s
logLang	17 (112)	4s	17 (112)	0% (0%)	4s	4s
neverlangJS	230 (1728)	✗	197 (1584)	14% (8%)	✗	20s
neverlangExpressions	80 (3537)	✗	72 (3489)	10% (1%)	✗	3m 25s

Table 3. Results of running `nlgcheck` on various pre-existing `Neverlang`-based projects, with and without optimizations, respectively. The **Reduction** column reports the % decrease in productions (nodes) from Optimization 1 alone, relative to the unoptimized CFG. An ✗ indicates that the process either crashed with an out-of-memory error or was manually aborted after 10 minutes. Optimization 1 (Section 4.5.1) reduces the number of productions and CFG nodes; Optimization 2 (Section 4.5.2) further improves execution time without further modifications to the graph.

be considered a false positive in the analysis performed by `nlgcheck`. Neither case strictly constitutes an error, as the compiler/interpreter would not crash, but we may consider extending `nlgcheck` to output a warning in these occurrences. Instead, *non-idiomatic attribute accesses* are actual errors that are beyond the analysis capabilities of `nlgcheck`. This is the only true limitation of our approach, which should therefore be considered fully sound only when writing *idiomatic* `Neverlang` code, at least until the tool is extended to capture all kinds of attribute accesses.

In mutation-testing terms, the *read without write*, *unnecessary copy*, and *dead code* mutants are *equivalent mutants* [53]: they do not change the behavior of the language on any program and therefore do not represent real errors. We did not classify each of the 194 undetected mutants into these categories, so we cannot report the exact number of equivalent mutants. Since the mutation score excludes equivalent mutants from the total number of mutants [53], the ratio of detected mutants reported in Table 2 (537 out of 731, 73%) is a lower bound on the mutation score of `nlgcheck`.

5.2 Performance Evaluation

Along with an evaluation of the ability of `nlgcheck` to detect errors, we are also interested in its performance. Static program analysis is notably intensive, and if the tool does not perform fast enough, it would be infeasible to use as part of the day-to-day development process. In fact, during the development of *nlgcheck*, it quickly became apparent that the base algorithm was only usable with very small languages. This led to the optimization presented in §4.5. In particular, in this section, we use *Optimization 1* to refer to graph deduplication (Section 4.5.1) and *Optimization 2* to refer to path deduplication (Section 4.5.2). This performance evaluation only compares `nlgcheck` against its own unoptimized and partially optimized variants. It does not compare against other static analysis tools for attribute grammars (*e.g.*, Silver, JastAdd, or Eli), which target a different analysis (flow-graph-based well-definedness rather than CFG-based path-sensitive analysis, §6) and are not directly comparable on this benchmark. The same holds on the functional side: we do not provide an empirical comparison of error-detection capability against these systems either, since doing so would require porting our mutation-testing benchmark onto AG systems with entirely different architectures and input languages. This remains future work.

To evaluate the final optimized implementation and the efficacy of the optimizations themselves, *nlgcheck* was tested with and without optimization on each language. The two optimizations can

also be disabled independently of each other. Evaluating Optimization 2 alone is left as future work, together with repeated measurements (§5.3).

Results. Table 3 summarizes the results of the performance evaluation. For each language, the second column reports the performance with both optimizations disabled. The third column yields the results with only Optimization 1 enabled, showing how the number of productions and of CFG nodes is reduced. Finally, in the last column, the tool is run with both optimizations enabled; since Optimization 2 only affects the visit algorithm and not the construction of the CFG itself, its structure and size remain the same as in the previous step.

We set a time limit of 10 minutes for each task, a time frame that we deemed reasonable to perform a full analysis of the language (for instance, before a daily commit). In most instances, the unoptimized or partially optimized attempts either kept running indefinitely (thus surpassing the time limit) or failed with an out-of-memory error.

In summary, Table 3 shows that while Optimization 1 slightly reduces the size of the CFG, in turn enabling the analysis on the previously failing *expr* case, it is not enough to make the tool suitable for use on real-world languages. The problem is eventually solved by adding Optimization 2, which makes the analysis run in a reasonable time for all the considered cases.

Expected time complexity. Optimization 1 (graph deduplication, Section 4.5.1) replaces each duplicate CFG fragment with a single shared copy. It does not solve graph isomorphism in general. Duplicates are detected by hashing, for each production, its nonterminals together with either the sequence of actions of its chain-shaped CFG or the identity of its semantic action (Section 4.5.1). Its expected cost is therefore linear in the total size of the productions and of the CFGs involved. Its benefit is proportional to how much syntactic duplication the grammar contains, *e.g.*, sibling productions differing only in an operator keyword. This explains why *logLang* sees no reduction at all (Table 3): it happens to contain no such duplicated fragments. Optimization 2 (path deduplication, Section 4.5.2) instead caches, for each nonterminal, the results already computed for a given head of the production context stack (§4.4.4). What would otherwise be a re-exploration of the same paths for every occurrence of a nonterminal thus becomes a single computation reused across all of them. Its benefit therefore scales with how often the same nonterminal recurs across productions. This is why it is responsible for the majority of the speedup on the larger, more repetitive *neverlangJS* and *neverlangExpressions* grammars.

5.3 Threats to Validity

We discuss the threats to the validity of this evaluation following the classification of Wohlin et al. [96].

Construct validity [96]. It is debatable whether the experiments performed within this evaluation accurately measure `nlgcheck`'s capability of detecting real errors (first experiment) and its performance (second experiment), respectively. For instance, we used only one kind of mutation operator—*i.e.*, changing an attribute name—whereas actual developer errors may vary. To counterbalance this limitation, we performed an extensive mutation testing experiment, thus mutating each attribute in the grammar at least once. We do not claim that mistyping an attribute name is the only mistake a `Neverlang` developer can make. Rather, among the many errors that can occur while developing a language, `nlgcheck` is concerned with only a very limited subset, and attribute-naming errors are the ones relevant to that subset. For instance, type errors are mostly irrelevant because they can be captured by the `Java` compiler without needing additional tooling. Similarly, errors such as forgetting an attribute or swapping two attributes can eventually be

remapped to a combination of attribute renames, as with separate compilation, there is no way to determine if two similar identifiers were meant to represent two different attributes or a typing error made by the developer while writing the same attribute twice.

We did not manually annotate each generated mutant with the specific error it was expected to trigger, which would have allowed us to compute `nlgcheck`'s precision and recall directly. We leave this as future work. Similarly, we did not separately quantify how many of the undetected mutants specifically fall under the *unnecessary copy* category discussed in §5.1; both would require re-instrumenting the mutation-testing campaign to retain a per-mutant classification, which we leave as future work alongside the precision/recall computation above. We also did not write test programs to confirm, by execution, that each detected mutant causes a runtime error. Each mutation replaces a character of an attribute name with a different random letter, which, with high probability, yields a name that no semantic action uses. A mutated read therefore has no matching write anywhere, and a mutated write may leave the reads of the original name without a matching write. Since a violation reported by `nlgcheck` lies on a valid path of the composed CFG, which corresponds to the traversal of an AST of the language, any execution following that path fails when it reaches the mutated access. The only exception is a path that requires a combination of branch outcomes that cannot occur at runtime, since `nlgcheck` does not track attribute values. Confirming each detected mutant with a concrete program is left as future work. The first experiment was likewise run once per language rather than repeated with different random seeds: since *nlgmutator*'s mutation operator exhaustively covers every attribute access expression in the grammar at least once by construction, a different seed would only change which character position and replacement letter are used to corrupt each already-determined site (§5), not which sites are exercised, so repetition would not meaningfully change detection coverage. Additionally, our experiments only introduce naming errors, not errors in the type hierarchy (*e.g.*, assigning an incompatible-but-similarly-named type), nor do they account for the fact that interpreters typically manage their runtime environment (the current state of execution) as an attribute. Our approach implicitly assumes that such environments are compatible across composed components, not just in terms of type but also of runtime behavior, a stronger assumption than the one we verify.

For the second experiment, each configuration (language, optimization level) was likewise run once, without a warm-up phase or repeated runs to report a median. We acknowledge that a single run per configuration cannot rule out incidental system-level noise (*e.g.*, just-in-time (JIT) compilation warm-up, garbage collection pauses) that repeated runs and a reported median would have controlled for. The problem of construct validity is less objective, as we arbitrarily determined a threshold of 10 minutes that some may consider either too large or too strict depending on their personal preference and development schedule. Eventually, we deemed this choice acceptable for two reasons:

- Our tool is not intended to be used on each compilation, but rather once or a few times a day, to ensure the correctness of the attribute grammar before performing a commit.
- The time limit was only relevant when the optimizations were not in place. All experiments made with both optimizations completed well within it. This is true even for a rather complex language, such as *neverlangJS*. The only outlier is *neverlangExpressions*, which was originally developed in a deliberately complex manner—*e.g.*, by using several abstractions and attribute mappings—to stress the modularization capabilities offered by `Neverlang`. We believe that for most real development scenarios, such as domain-specific languages, the 10-minute target should never be reached, and possibly `nlgcheck` could even be included as a standard compilation phase—at least for languages in the 3 to 4-second range.

Internal validity [96]. Some of the experiments may be affected by our specific expertise with `Neverlang` and with languages developed by its means. To avoid this threat as much as possible, we included several different languages in the experimentation, making sure to include domain-specific languages, general-purpose languages, and sublanguages, each preceding the existence of `nlgcheck`, which should mitigate the risk of having designed languages that specifically fit `nlgcheck` and its requirements. All five languages were nonetheless developed within our own research group, with no external contributors. For instance, *stateMachines* was originally authored as teaching material for a hands-on session at the BISS 2023 summer school, independently of and prior to this evaluation, but by a member of our group rather than an external developer. We also included results both with and without optimization to avoid any concerns about bias in the results.

External validity [96]. Even though we tried to design the static analysis approach and its proof in a general manner that should be suitable for any language workbench, it is undeniable that the idea for this work originated from a `Neverlang`-specific need, and the solution was then designed according to these needs. For instance, while we believe we adequately justified the advantages of separate compilation in §1, it must be said that separate compilation is far from common in language workbenches: `Neverlang` is, to the best of our knowledge, the only one that fully supports the compilation of each individual artifact separately. This should not be intended as a limitation of other language workbenches, but rather a design choice. Most language workbenches reasonably opt for relaxing the requirement of separate compilation in favor of the guarantees provided by joint compilation, such as checking attribute types and dependencies. We can only hope that this contribution encourages other language workbenches trying to support separate compilation to achieve the best of both worlds.

6 Related Work

The challenge of ensuring static correctness in AGs has been a cornerstone of language engineering since Knuth's seminal work [61]. However, as language workbenches have evolved toward modularity and separate compilation, a significant gap has emerged between structural completeness and operational safety. This section situates `nlgcheck` within the landscape of modular AG verification, advanced AG formalisms, and static analysis techniques.

Structural Verification vs. Operational Path-Sensitivity. The primary focus of traditional AG verification is *well-definedness*—ensuring that for any valid syntax tree, every attribute instance has a unique definition and no circular dependencies exist [36].

Modern systems like Silver [57, 89] have pioneered *modular well-definedness analysis*. This allows independent language extensions to be verified in isolation, relying only on information from their dependencies [57]. JastAdd [37, 49], instead, weaves all the aspects of a specification together before generating the evaluator. Its optional check for missing inherited equations therefore applies to the complete specification, and, as observed by Kaminski and Van Wyk [57], missing equations are otherwise reported when attributes are evaluated. It is important to note, however, that in both cases the modularity operates at the specification level: all contributing modules must still be present, in source form, at the single joint compilation step that generates the resulting language processor—as with the other systems discussed throughout this section (§5 elaborates on why `Neverlang`'s genuine separate compilation of already-compiled binary components is, to our knowledge, uncommon among language workbenches). While these tools are highly effective at ensuring structural integrity (i.e., that a definition exists in the codebase) within such a joint compilation, they treat the internal logic of semantic actions as *black boxes*.

In a framework like `Neverlang`, where semantic actions are implemented via imperative `Java` code and attribute access is mediated by dynamic maps, the existence of a definition does not guarantee runtime safety. Our tool addresses this by shifting the focus from "is the attribute defined in the module?" to "is the attribute initialized before use along every possible execution path?" In an AG specified with equations, the first question suffices: the evaluator schedules each equation so that the attributes it uses are computed first, either statically, as with the visit sequences of ordered AGs [59], or on demand, as in JastAdd [49]. In `Neverlang`, instead, the order in which attributes are written and read is fixed by the imperative code of the semantic actions and by the `eval` calls written by the developer (§3.1). A read may therefore precede the write that would define the attribute, even if such a write exists somewhere in the composition. The path-sensitive data-flow analysis of §4 is what prevents the resulting *MissingAttr* runtime error in this setting.

Flow Graphs versus Control/Data-Dependency Graphs. Classical AG well-definedness analyses, from Knuth's original proposal [61] to ordered attribute grammars [59] and their implementation in systems such as Eli [46] and Silver [57, 89], are built around a *flow graph* per production: a graph whose vertices are the attributes occurring on the nonterminals of that production, with an edge between two attributes whenever one is declared, by the production's equations, to depend on the other. Our tool, in contrast, builds a *control/data-dependency graph* (the CFG of §4) from the compiled bytecode of each semantic action, whose vertices are individual read, write, and branch instructions instead of whole attributes.

We do not claim that this finer granularity makes our analysis more precise than the classical ones on AGs specified with equations. `nlgcheck` does not track attribute values. In the semantic action of Listing 5, it therefore requires both `Op[2].val` and `Op[3].val` to be defined, exactly as a flow-graph-based analysis of the equation `Op.val = Op[1].val ? Op[2].val : Op[3].val` would. We are not aware of a realistic AG that can be specified both with equations and with `Neverlang` semantic actions, is well defined for all syntax trees, fails the classical analysis, and passes ours. The control/data-dependency graph is needed, instead, because of how `Neverlang` semantic actions are written and compiled:

(1) the evaluation order is written by the developer through `eval`, so well-definedness depends on the order in which reads and writes occur, which a flow graph abstracts away;
(2) attributes are mutable locations that may be written several times (§4.2);
(3) the analysis must run on compiled binaries, after composition, where no equations are available.

Two examples illustrate the first two points. In the first, a semantic action for `Op` reads `$1.val` before evaluating its first child with `eval $1`. Read as an equation for `Op.val`, the action is complete and non-circular, so an analysis of its declared dependencies accepts it, although every execution fails. `nlgcheck`, instead, reports a *MissingAttr* violation, since no path leading to the read has written `Op[1].val`. In the second example, an environment is threaded through a sequence of statements by overwriting a single attribute `env`. The action of a production `Stmts <- Stmt Stmts` writes `$1.env` from `$0.env` and evaluates `$1`. It then copies `$1.env`, possibly updated by the action of `Stmt`, into `$2.env`, evaluates `$2`, and finally copies `$2.env` back into `$0.env`. Read as equations, `env` would be both inherited and synthesized by the same nonterminals, so the specification is not a well-formed AG and a classical system rejects it. `nlgcheck` accepts it, since on every path each read of `env` is preceded by a write. The second example shows a difference in representation, not an increase in precision: a specification with equations would use two distinct attributes, such as `env_in` and `env_out`, and would pass the classical analysis.

We do not provide an empirical performance comparison against flow-graph-based tools such as Eli or Silver. As §5.2 already notes, our performance evaluation only compares `nlgcheck` against

its own unoptimized and partially optimized variants, since these systems target a structurally different analysis on different inputs (attribute-level flow graphs derived from AG equations, as opposed to a CFG derived from compiled bytecode) and are not directly comparable on a shared benchmark.

The Modularity Paradox: Flexibility vs. Static Safety. A recurring theme in the literature is the trade-off between the expressiveness of AGs and the ease of their verification.

Extensions such as reference attribute grammars [48] and higher-order attribute grammars [91] significantly increase the power of AGs by allowing the tree structure to be computed dynamically. However, systems implementing these (e.g., JastAdd) typically enforce strict static typing and require global knowledge of the tree structure to ensure safety [37].

Neverlang adopts a *feature-oriented* architecture designed for true separate compilation and binary distribution. This requires a loose coupling where attributes are not hard-coded offsets but dynamic entries in a map. While this maximizes modularity—allowing users to swap components without recompilation—it circumvents the safety nets of traditional compilers.

Mernik et al. [73] suggest that the power of domain-specific tools often necessitates a reliance on strict static discipline. Our tool serves as the essential compensatory mechanism for Neverlang's dynamic nature. By applying static analysis to the generated bytecode, it restores the safety guarantees of a strongly typed system to a highly decoupled, dynamic architecture.

Evolution of Modularity: From Composable Attribute Grammars to Forwarding. The quest for reusability has led to several modular composition techniques:

Early work on composable attribute grammars [40] enabled the combination of smaller, independent AGs to solve complex subproblems (e.g., name analysis). Similarly, Generic Attribute Grammars [60] focused on decomposing AGs into reusable components. Ganzinger [43] pursued analogous goals from the compiler-construction side, increasing the modularity and language-independence of automatically generated compilers, while Polyglot [75] offers an extensible compiler framework for Java built around a similar notion of composable language extensions. *Forwarding* [90] provides a mechanism for a node to forward attribute queries to a replacement subtree, aiding in modular extensibility.

Beyond AG-specific systems, general-purpose language workbenches address modular language composition at a coarser granularity. MontiCore [17, 18, 64] composes languages from independent syntactic and semantic features. JetBrains MPS (Meta Programming System) [92] supports modularizing both a language and its IDE tooling. Melange [33] treats language components as reusable, composable meta-language artifacts. Leduc et al. [67] pursue a related goal with *Alex*, a non-intrusive approach to modularizing language concerns behind well-defined interfaces, built atop the Eclipse Modeling Framework. All four, like the systems discussed above, compose language definitions before generating the resulting compiler or interpreter, rather than composing already-compiled binary artifacts as `nlgcheck` does.

While these approaches facilitate design-time modularity, they often struggle with deployment-time modularity (binary distribution). They typically require the entire grammar to be re-analyzed or re-compiled when modules are composed. Our tool is unique in that it validates the interaction of separately compiled binaries by analyzing the underlying data-flow of their semantic actions, ensuring that modular composition does not lead to runtime failures.

Data-Flow Analysis and Fixed-Point Iteration. The methodology of `nlgcheck` draws from classic compiler optimization and verification techniques [1].

Tools like the Extended Static Checker for Java (ESC/Java) [41] demonstrated that data-flow analysis could effectively catch null pointer exceptions and array-out-of-bounds errors in general-purpose code. Our approach applies these principles specifically to the domain of AGs.

Historically, Farrow used fixed-point evaluators to ensure value convergence in circular AGs [39]. Our tool repurposes fixed-point principles for state-space bounding. By utilizing the Soot framework [87] to analyze Java bytecode, `nlgcheck` performs an iterative analysis over the CFG. To ensure termination in the presence of recursive semantic actions or complex loops, it implements a recursion depth bound, making the analysis of infinite execution paths computationally tractable.

Loops within the imperative body of a single semantic action (as opposed to recursion across semantic actions, which the depth bound of §4.4 addresses) are handled implicitly by the underlying CFG traversal: since `nlgcheck` visits the graph rather than executing it, a loop is simply a cycle that the visit algorithm's already-visited-state tracking (Constraint (b), §4.4) prevents from being traversed indefinitely, in the same way classical data-flow frameworks iterate over loops until a fixed point is reached [1], rather than unrolling them.

7 Conclusions and Future Work

In this work, we presented a novel approach to overcome the limitations of separate compilation in language workbenches. Separate compilation offers several advantages regarding modularity and the distribution of pre-compiled artifacts through public repositories. However, this requirement is often relaxed to comply with other correctness guarantees and non-functional properties [68]. With this contribution, we aim to bridge this gap, so that future language workbenches can adopt separate compilation without giving up the detection of attribute errors before runtime. Under separate compilation, undefined and ill-typed attribute accesses are then detected when the components are composed, instead of at the runtime of the compiler, while repairing them relies on the composition mechanisms discussed in §4.6. Concretely, this paper contributes: (i) a formal model of semantic actions as control/data-dependency graphs, built directly from separately compiled bytecode rather than from source-level specifications; (ii) a static analysis, together with a soundness proof, that detects undefined and ill-typed attribute accesses along every possible execution path, including across recursive compositions of separately compiled language components; (iii) two optimizations (graph and path deduplication, §4.5) that make this analysis practical on non-trivial languages; and (iv) `nlgcheck`, an implementation of this approach for `Neverlang`, together with an empirical evaluation of its error-detection capability and performance on several pre-existing `Neverlang`-based languages. The static analysis approach presented in this paper is sound and effective in practice, and the optimizations mentioned above make it usable on a daily basis during the development process.

Several improvements could expand this research. First, the current analysis is limited to a single AST visit at a time: `nlgcheck` checks each role's CFG G_r independently, resetting the production context stack between roles (roles combined with the `:` operator of §3.1 are executed in a single traversal and are therefore analyzed together)—in other words, `nlgcheck`'s current model assumes the checked artifact behaves as a single-pass compiler. This is a hard limitation whenever a later role in a language's **`roles`** clause (§3.1) depends on attributes set by an earlier one, as when, for instance, a `permissions` role executes after, and relies on, a `terminal-evaluation` role: `nlgcheck` currently cannot verify that an attribute set by an earlier role is available to a later one, as the state from the first traversal is reset before the second begins.

In practice, we expect this limitation to matter less than dependencies within a role. Roles correspond to compiler phases and are coarse grained, and their order is declared explicitly by the **`roles`** clause, so a dependency between two roles follows the order of the phases and is easy to manage without an analysis. The languages evaluated in §5 confirm this. In *logLang*, the

`terminal-evaluation` role writes the values of terminals (the `string` and `identifier` attributes), which the later `logging`, `permissions`, and `execution` roles read. In *stateMachines*, the `id` attribute written by the `evaluation` role is read by the later `register` role. In *numericUnitExpr* and *neverlangJS*, no attribute written by one role is read by another: in *numericUnitExpr*, for instance, the `unit-checking` role only uses the `label` attribute, while the `evaluation` role only uses `value`. None of these languages exhibited a runtime error at a role boundary, so the limitation remains, for now, a theoretical rather than an observed one.

Other minor improvements include better error reporting, such as warnings for issues currently undetected by `nlgcheck`. In this regard, so-called "non-idiomatic" attribute accesses should either trigger a warning or, even better, be included as part of the analysis. Finally, this approach should be tested with other language workbenches to verify the degree to which the solutions proposed in this work are limited to `Neverlang` and its ecosystem.

Acknowledgments

A special thanks to Gianluca Nitti for his support in the development of `nlgcheck`.

References

[1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. 2006. *Compilers: Principles, Techniques, and Tools* (second ed.). Addison-Wesley, Boston, MA, USA.

[2] Alfred V. Aho, Ravi Sethi, and Jeffrey D. Ullman. 1986. *Compilers: Principles, Techniques, and Tools*. Addison Wesley, Reading, Massachusetts.

[3] Alfred V. Aho and Jeffrey D. Ullman. 1972. *The Theory of Parsing, Translation, and Compiling*. Prentice-Hall, Inc.

[4] Frances E. Allen. 1970. Control Flow Analysis. In *Proceedings of a Symposium on Compiler Optimization (SCO'70)*, Robert S. Northcote (Ed.). ACM, Urbana-Champaign, IL, USA, 1–19.

[5] John Randal Allen. 1983. *Dependence Analysis for Subscripted Variables and Its Application to Program Transformations*. PhD Thesis. Rice University, Houston, TX, USA.

[6] John R. Allen, Ken Kennedy, Carrie Porterfield, and Joe Warren. 1983. Conversion of Control Dependence to Data Dependence. In *Proceedings of the 10th Symposium on Principles of Programming Languages (POPL'83)*, Alan Demers and Tim Teitelbaum (Eds.). ACM, Austin, TX, USA, 177–189.

[7] Utpal Banerjee. 1979. *Speedup of Ordinary Programs*. PhD Thesis. University of Illinois at Urbana-Champaign, Urbana-Champaign, IL, USA.

[8] Utpal Banerjee. 1988. An Introduction to a Formal Theory of Dependence Analysis. *Journal of Supercomputing* 2 (Oct. 1988), 133–149.

[9] Arthur J. Bernstein. 1966. Analysis of Programs for Parallel Processing. *IEEE Transactions on Electronic Computers* 15, 5 (Oct. 1966), 757–763.

[10] Francesco Bertolotti, Walter Cazzola, and Luca Favalli. 2023. On the Granularity of Linguistic Reuse. *Journal of Systems and Software* 202 (Aug. 2023). doi:10.1016/j.jss.2023.111704

[11] George McArthur Beshers and Roy Harold Campbell. 1985. Maintained and Constructor Attributes. In *Proceedings of the Symposium on Language Issues in Programming Environments (SLIPE'85)*, Teri F. Payton, L. Peter Deutsch, and James Purtilo (Eds.). ACM, Seattle, WA, USA, 34–42.

[12] Eric Bodden. 2012. Inter-Procedural Data-Flow Analysis with IFDS/IDE and Soot. In *Proceedings of the 1st International Workshop on State of the Art in Java Program Analysis (SOAP'12)*, Eric Bodden, Laurie Hendren, and Patrick Lam (Eds.). ACM, Beijing, China, 3–8.

[13] John Boyland. 1998. Analyzing Direct Non-local Dependencies in Attribute Grammars. In *Proceedings of the 7th International Conference on Compiler Construction (CC'98) (LNCS 1383)*, Kai Koskimies (Ed.). Springer, Lisbon, Portugal, 31–49.

[14] Federico Bruzzone and Walter Cazzola. 2026. A Multi-Dimensional, Per-Pass Empirical Study of the LLVM Optimization Pipeline. arXiv:2606.31238 [cs.SE] https://arxiv.org/abs/2606.31238

[15] Federico Bruzzone, Walter Cazzola, and Luca Favalli. 2026. Code Less to Code More: Streamlining Language Server Protocol and Type System Development for Language Families. *Journal of Systems and Software* 231 (Jan. 2026). doi:10.1016/j.jss.2025.112554

[16] Federico Bruzzone, Walter Cazzola, and Luca Favalli. 2026. Generalized Software Product Line Extraction. arXiv:2605.28989 [cs.SE] https://arxiv.org/abs/2605.28989

[17] Arvid Butting, Robert Eikermann, Oliver Kautz, Bernhard Rumpe, and Andreas Wortmann. 2018. Controlled and Extensible Variability of Concrete and Abstract Syntax with Independent Language Features. In *Proceedings of the 12th International Workshop on Variability Modelling of Software Intensive Systems (VAMOS'18)*. ACM, Madrid, Spain, 75–82.

[18] Arvid Butting, Robert Eikermann, Oliver Kautz, Bernhard Rumpe, and Andreas Wortmann. 2019. Systematic Composition of Independent Language Features. *Journal of Systems and Software* 152 (June 2019), 50–69.

[19] David Callahan, Keith D. Cooper, Ken Kennedy, and Linda Torczon. 1986. Interprocedural Constant Propagation. In *Proceedings of the Sigplan Symposium on Compiler Construction (SCC'86)*, Richard L. Wexelblat (Ed.). ACM, Palo Alto, CA, USA, 152–161.

[20] Walter Cazzola. 2012. Domain-Specific Languages in Few Steps: The Neverlang Approach. In *Proceedings of the 11th International Conference on Software Composition (SC'12) (Lecture Notes in Computer Science 7306)*, Thomas Gschwind, Flavio De Paoli, Volker Gruhn, and Matthias Book (Eds.). Springer, Prague, Czech Republic, 162–177.

[21] Walter Cazzola and Luca Favalli. 2022. Towards a Recipe for Language Decomposition: Quality Assessment of Language Product Lines. *Empirical Software Engineering* 27, 4 (April 2022). doi:10.1007/s10664-021-10074-6

[22] Walter Cazzola and Luca Favalli. 2023. The Language Mutation Problem: Leveraging Language Product Lines for Mutation Testing of Interpreters. *Journal of Systems and Software* 195 (Jan. 2023). doi:10.1016/j.jss.2022.111533

[23] Walter Cazzola, Paola Giannini, and Albert Shaqiri. 2016. Formal Attributes Traceability in Modular Language Development Frameworks. *Electronic Notes In Theoretical Computer Science* 322 (April 2016), 119–134. doi:10.1016/j.entcs.2016.03.009

[24] Walter Cazzola and Albert Shaqiri. 2016. Modularity and Optimization in Synergy. In *Proceedings of the 15th International Conference on Modularity (Modularity'16)*, Don Batory (Ed.). ACM, Málaga, Spain, 70–81.

[25] Walter Cazzola and Edoardo Vacchi. 2013. Neverlang 2: Componentised Language Development for the JVM. In *Proceedings of the 12th International Conference on Software Composition (SC'13) (Lecture Notes in Computer Science 8088)*, Walter Binder, Eric Bodden, and Welf Löwe (Eds.). Springer, Budapest, Hungary, 17–32.

[26] Walter Cazzola and Edoardo Vacchi. 2016. Language Components for Modular DSLs using Traits. *Computer Languages, Systems & Structures* 45 (April 2016), 16–34. doi:10.1016/j.cl.2015.12.001

[27] Keith D. Cooper, Timothy J. Harvey, and Ken Kennedy. 2006. *A Simple, Fast Dominance Algorithm.* Technical Report TR-06-33870. Rice University, Houston, TX, USA.

[28] Patrick Cousot. 1996. Abstract Interpretation. *ACM Computing Surveys* 28, 2 (June 1996), 324–328.

[29] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. 1989. An Efficient Method of Computing Static Single Assignment Form. In *Proceedings of the 16th Symposium on Principles of Programming Languages (POPL'89)*. ACM, Austin, TX, USA, 25–35.

[30] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. 1991. Efficiently Computing Static Single Assignment Form and the Control Dependence Graph. *ACM Transactions on Programming Languages and Systems* 13, 4 (Oct. 1991), 451–490.

[31] Andreas Dann, Ben Hermann, and Eric Bodden. 2019. SootDiff: Bytecode Comparison Across Different Java Compilers. In *Proceedings of the 8th International Workshop on State of the Art in Java Program Analysis (SOAP'12)*, Neville Grech and Thierry Lavoie (Eds.). ACM, Phoenix, AZ, USA, 14–19.

[32] Oege de Moor, Kevin Backhouse, and S. Doaitse Swierstra. 2000. First-Class Attribute Grammars. *Informatica: An International Journal of Computing and Informatics* 24, 2 (April 2000), 329–341.

[33] Thomas Degueule, Benoît Combemale, Arnaud Blouin, Olivier Barais, and Jean-Marc Jézéquel. 2015. Melange: a Meta-Language for Modular and Reusable Development of DSLs. In *Proceedings of the 8th International Conference on Software Language Engineering (SLE'15)*, Davide Di Ruscio and Markus Völter (Eds.). ACM, Pittsburgh, PA, USA, 25–36.

[34] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. 1978. Hints on Test Data Selection: Help for the Practicing Programmer. *IEEE Computer* 11, 4 (April 1978), 34–41.

[35] Pierre Deransart, Martin Jourdan, and Bernard Lorho. 1984. Speeding Up Circularity Tests for Attribute Grammars. *Acta Informatica* 21, 4 (1984), 375–391.

[36] Pierre Deransart, Martin Jourdan, and Bernard Lorho. 1988. *Attribute Grammars: Definitions, Systems and Bibliography.* Springer.

[37] Torbjörn Ekman and Görel Hedin. 2007. The JastAdd System — Modular Extensible Compiler Construction. *Science of Computer Programming* 69, 1-3 (Dec. 2007), 14–26.

[38] Sebastian Erdweg, Paolo G. Giarrusso, and Tillmann Rendel. 2012. Language Composition Untangled. In *Proceedings of the 12th Workshop on Language Description, Tools, and Applications (LDTA'12)*, Anthony M. Sloane and Suzana Andova (Eds.). ACM, Tallinn, Estonia.

[39] Rodney Farrow. 1986. Automatic Generation of Fixed-Point-Finding Evaluators for Circular, but Well-Defined, Attribute Grammars. In *Proceedings of the Symposium on Compiler Construction (SCC'86)*, Richard L. Wexelblat (Ed.). ACM, Palo Alto, CA, USA, 85–98.

[40] Rodney Farrow, Thomas J. Marlowe, and Daniel M. Yellin. 1992. Composable Attribute Grammars: Support for Modularity in Translator Design and Implementation. In *Proceedings of the 19th ACM Symposium on Principles of Programming Languages (POPL'92)*, Ravi Sethi (Ed.). ACM, Albuquerque, NM, USA, 223–234.
[41] Cormac Flanagan, K. Rustan M. Leino, Mark Lillibridge, Greg Nelson, James B. Saxe, and Raymie Stata. 2002. Extended Static Checking for Java. In *Proceedings of the 23rd Conference on Programming Language Design and Implementation (PLDI'02)*, Laurie J. Hendren (Ed.). ACM, Berlin, Germany, 234–245.
[42] Martin Fowler. 2005. Language Workbenches: The Killer-App for Domain Specific Languages? Martin Fowler's Blog. http://www.martinfowler.com/articles/languageWorkbench.html
[43] Harald Ganzinger. 1983. Increasing Modularity and Language-Independency in Automatically Generated Compilers. *Science of Computer Programming* 3, 3 (Dec. 1983), 223–278.
[44] Loukas Georgiadis, Giuseppe F Italiano, Luigi Laura, and Federico Santaroni. 2012. An Experimental Study of Dynamic Dominators. In *Proceedings of the 20th European Symposium on Algorithms (ESA'12) (LNCS 7501)*, Leah Epstein and Paolo Ferragina (Eds.). Springer, Ljubljana, Slovenia, 491–502.
[45] Loukas Georgiadis, Robert E. Tarjan, and Renato F. Werneck. 2006. Finding Dominators in Practice. *Journal of Graph Algorithms and Applications* 10, 1 (Jan. 2006), 69–94.
[46] Robert W. Gray, Steven P. Levi, Vincent P. Heuring, Anthony M. Sloane, and William M. Waite. 1992. Eli: A Complete, Flexible Compiler Construction System. *Commun. ACM* 35, 2 (Feb. 1992), 121–130.
[47] Görel Hedin. 1994. An Overview of Door Attribute Grammars. In *Proceedings of the 5th International Conference on Compiler Construction (CC'94)*, Peter Fritzon (Ed.). Springer, Edinburgh, United Kingdom, 31–51.
[48] Görel Hedin. 2000. Reference Attributed Grammars. *Informatica: An International Journal of Computing and Informatics* 24, 2 (April 2000), 31–51.
[49] Görel Hedin and Eva Magnusson. 2003. JastAdd — An Aspect-Oriented Compiler Construction System. *Science of Computer Programming* 47, 1 (April 2003), 37–58.
[50] John L. Hennessy and David A. Patterson. 2011. *Computer Architecture: A Quantitative Approach* (fifth edition ed.). Morgan Kaufmann.
[51] Roger Hoover. 1987. *Incremental Graph Evaluation.* Technical Report TR87-836. Cornell University, Ithaca, NY, USA.
[52] Mehdi Jazayeri, William F. Ogden, and William C. Rounds. 1975. The Intrinsically Exponential Complexity of the Circularity Problem for Attribute Grammars. *Commun. ACM* 18, 12 (Dec. 1975), 697–706.
[53] Yue Jia and Mark Harman. 2011. An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering* 37, 5 (Sept. 2011), 649–678.
[54] Gregory F. Johnson and Charles N. Fischer. 1982. Non-Syntactic Attribute Flow in Language Based Editors. In *Proceedings of the 9th Symposium on Principles of Programming Languages (POPL'82)*, Richard A. DeMillo (Ed.). ACM, Albuquerque, NM, USA, 185–195.
[55] Gregory F. Johnson and Charles N. Fischer. 1985. A Meta-Language and System for Nonlocal Incremental Attribute Evaluation in Language-Based Editors. In *Proceedings of the 12th Symposium on Principles of Programming Languages (POPL'85)*, Mary S. Van Deusen, Zvi Galil, and Brian K. Reid (Eds.). ACM, New Orleans, LA, USA, 141–151.
[56] Gail Elaine Kaiser. 1985. *Semantics for Structure Editing Environments.* PhD Thesis. Carnegie Mellon University, Pittsburgh, PA, United States.
[57] Ted Kaminski and Eric Van Wyk. 2013. Modular Well-Definedness Analysis for Attribute Grammars. In *Proceedings of the 5th International Conference on Software Language Engineering (SLE'13) (Lecture Notes in Computer Science 7745)*, Krzysztof Czarnecki and Görel Hedin (Eds.). Springer, Dresden, Germany, 352–371.
[58] Kadiray Karakaya, Stefan Schott, Jonas Klauke, Eric Bodden, Markus Schmidt, Linghui Luo, and Dongjie He. 2024. SootUp: A Redesign of the Soot Static Analysis Framework. In *Proceedings of the 30th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS'24) (LNCS 14570)*, Bernd Finkbeiner and Laura Kovács (Eds.). Springer, Luxembourg City, Luxembourg, 229–247.
[59] Uwe Kastens. 1980. Ordered Attribute Grammars. *Acta Informatica* 13, 3 (March 1980), 229–256.
[60] Uwe Kastens and William M. Waite. 1994. Modularity and Reusability in Attribute Grammars. *Acta Informatica* 31, 7 (July 1994), 601–627.
[61] Donald E. Knuth. 1968. Semantics of Context-Free Languages. *Mathematical Systems Theory* 2, 2 (1968), 127–145.
[62] Donald E. Knuth. 1971. Semantics of Context-Free Languages: Correction. *Mathematical Systems Theory* 5, 1 (1971), 95–96. doi:10.1007/BF01702865
[63] Donald E. Knuth. 1990. The Genesis of Attribute Grammars. In *Proceedings of the International Conference on Attribute Grammars and Their Applications (WAGA'90) (Lecture Notes in Computer Science 461)*, Pierre Deransart and Martin Jourdan (Eds.). Springer, Paris, France, 1–12.
[64] Holger Krahn, Bernhard Rumpe, and Steven Völkel. 2010. MontiCore: A Framework for Compositional Development of Domain Specific Languages. *International Journal on Software Tools for Technology Transfer* 12, 5 (Sept. 2010), 353–372.

[65] Thomas Kühn, Walter Cazzola, Nicola Pirritano Giampietro, and Massimiliano Poggi. 2019. Piggyback IDE Support for Language Product Lines. In *Proceedings of the 23rd International Software Product Line Conference (SPLC'19)*, Thomas Thüm and Laurence Duchien (Eds.). ACM, Paris, France, 131–142.
[66] Patrick Lam, Eric Bodden, Ondrej Lhoták, and Laurie Hendren. 2011. The Soot Framework for Java Program Analysis: A Retrospective. In *Proceedings of the Cetus Users and Compiler Infastructure Workshop (CETUS'11)*, Sam Midkiff, Rudi Eigenmann, and Hansang Bae (Eds.). Purdue University, Galveston Island, TX, USA, 1–8.
[67] Manuel Leduc, Thomas Degueule, and Benoît Combemale. 2018. Modular Language Composition for the Masses. In *Proceedings of the 11th International Conference on Software Language Engineering (SLE'18)*, Tanja Mayerhofer and Friedrich Steimann (Eds.). ACM, Boston, MA, USA, 47–59.
[68] Manuel Leduc, Thomas Degueule, Eric Van Wyk, and Benoît Combemale. 2020. The Software Language Extension Problem. *Software and Systems Modeling* 19, 2 (Jan. 2020), 263–267.
[69] Thomas Lengauer and Robert Endre Tarjan. 1979. A Fast Algorithm for Finding Dominators in a Flowgraph. *ACM Transactions on Programming Languages and Systems* 1, 1 (Jan. 1979), 121–141.
[70] Ondřej Lhoták and Laurie Hendren. 2003. Scaling Java Points-to Analysis Using SPARK. In *Proceedings of the 12th International Conference on Compiler Construction (CC'03) (LNCS 2622)*, Görel Hedin (Ed.). Springer, Warsaw, Poland, 153–169.
[71] Edward S. Lowry and C. W. Medlock. 1969. Object Code Optimization. *Commun. ACM* 12, 1 (Jan. 1969), 13–22.
[72] David Méndez-Acuña, José A. Galindo, Thomas Degueule, Benoît Combemale, and Benoît Baudry. 2016. Leveraging Software Product Lines Engineering in the Development of External DSLs: A Systematic Literature Review. *Computer Languages, Systems & Structures* 46 (Nov. 2016), 206–235.
[73] Marjan Mernik, Jan Heering, and Anthony M. Sloane. 2005. When and How to Develop Domain Specific Languages. *Comput. Surveys* 37, 4 (Dec. 2005), 316–344.
[74] Jerome Miecznikowski. 2003. *New Algorithms for a Java Decompiler and Their Implementation in Soot.* Master's Thesis. McGill University, Montrèal, QB, Canada.
[75] Nathaniel Nystrom, Michael R. Clarkson, and Andrew C. Myers. 2003. Polyglot: An Extensible Compiler Framework for Java. In *Proceedings of the 12th International Conference on Compiler Construction (CC'03) (Lecture Notes in Computer Science 2622)*. Springer, Warsaw, Poland, 138–152.
[76] Jukka Paakki. 1995. Attribute Grammar Paradigms: A High-Level Methodology in Language Implementation. *ACM Computer Survey* 27, 2 (June 1995), 196–255.
[77] Patrice Pominville, Feng Qian, Raja Vallée-Rai, Laurie Hendren, and Clark Verbrugge. 2001. A Framework for Optimizing Java Using Attributes. In *Proceedings of the 10th International Conference on Compiler Construction (CC'01) (LNCS 2027)*, Reinhard Wilhelm (Ed.). Springer, Genoa, Italy, 334–354.
[78] Reese T. Prosser. 1959. Applications of Boolean Matrices to the Analysis of Flow Diagrams. In *Proceedings of the Eastern Joint IRE-AIEE-ACM Computer Conference (IRE-AIEE-ACM'59)*, Frank E. Heart (Ed.). ACM, Boston, MA, USA, 133–138.
[79] Paul W. Purdom Jr and Edward F. Moore. 1972. Immediate Predominators in a Directed Graph. *Commun. ACM* 15, 8 (Aug. 1972), 777–778.
[80] Thomas Reps, Susan Horwitz, and Mooly Sagiv. 1995. Precise Interprocedural Dataflow Analysis via Graph Reachability. In *Proceedings of the 22nd Symposium on Priciples of Programming Languages (POPL'95)*, Ron K. Cytron and Peter Lee (Eds.). ACM, San Francisco, CA, USA, 49–61.
[81] Nathan Ringo, Lucas Kramer, and Eric Van Wyk. 2023. Nanopass Attribute Grammars. In *Proceedings of the 16th International Conference on Software Language Engineering (SLE'23)*, Thomas Degueule and Elizabeth Scott (Eds.). ACM, Cascais, Portugal, 70–83.
[82] Algis Rudys and Dan S. Wallach. 2002. Enforcing Java Run-Time Properties Using Bytecode Rewriting. In *Proceedings of the International Conference on Software Security: Theories and Systems (ISSS'02) (LNCS 2609)*, Mitsuhiro Okada, Benjamin C. Pierce, André Scedrov, Hideyuki Tokuda, and Akinori Yonezawa (Eds.). Springer, Tokyo, japan, 185–200.
[83] Mooly Sagiv, Thomas Reps, and Susan Horwitz. 1996. Precise Interprocedural Dataflow Analysis with Applications to Constant Propagation. *Theoretical Computer Science* 167, 1-2 (Oct. 1996), 131–170.
[84] Vijay Sundaresan, Laurie Hendren, Chrislain Razafimahefa, Raja Vallée-Rai, Patrick Lam, Etienne Gagnon, and Charles Godin. 2000. Practical Virtual Method Call Resolution for Java. In *Proceedings of the 15th Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA'00)*, Mary Beth Rosson and Doug Lea (Eds.). ACM, Minneapolis, MN, USA, 264–280.
[85] Ross Albert Towle. 1976. *Control and Data Dependence for Program Transformations.* PhD Thesis. University of Illinois at Urbana-Champaign, Urbana-Champaign, IL, USA.
[86] Edoardo Vacchi and Walter Cazzola. 2015. Neverlang: A Framework for Feature-Oriented Language Development. *Computer Languages, Systems & Structures* 43, 3 (Oct. 2015), 1–40. doi:10.1016/j.cl.2015.02.001
[87] Raja Vallée-Rai, Phong Co, Etienne Gagnon, Laurie Hendren, Patrick Lam, and Vijay Sundaresan. 1999. Soot: A Java Bytecode Optimization Framework. In *Proceedings of the Conference of the Centre for Advanced Studies on Collaborative*

Research (CASCON'99), Stephen A. MacKay and J. Howard Johnson (Eds.). IBM Press, Mississauga, ON, Canada, 1–11.

[88] Raja Vallée-Rai, Etienne Gagnon, Laurie J. Hendren, Patrick Lam, Patrice Pominville, and Vijay Sundaresan. 2000. Optimizing Java Bytecode Using the Soot Framework: Is It Feasible?. In *Proceedings of the 9th International Conference on Compiler Construction (CC'00) (LNCS 1781)*, David A. Watt (Ed.). Springer, Berlin, Germany, 18–34.

[89] Eric Van Wyk, Derek Bodin, Jimin Gao, and Lijesh Krishnan. 2010. Silver: an Extensible Attribute Grammar System. *Science of Computer Programming* 75, 1-2 (Jan. 2010), 39–54.

[90] Eric Van Wyk, Oege de Moor, Kevin Backhouse, and Paul Kwiatkowski. 2002. Forwarding in Attribute Grammars for Modular Language Design. In *Proceedings of the 11th International Conference on Compiler Construction (CC'02) (LNCS 2304)*, R. Nigel Horspool (Ed.). Springer, Grenoble, France, 128–142.

[91] Harald H. Vogt, S. Doaitse Swierstra, and Matthijs F. Kuiper. 1989. Higher Order Attribute Grammars. In *Proceedings of the Conference on Programming Language Design and Implementation (PLDI'89)*, Richard L. Wexelblat (Ed.). Portland, OR, USA, 131–145.

[92] Markus Völter. 2011. Language and IDE Modularization and Composition with MPS. In *Proceedings of the 4th International Summer School on Generative and Transformational Techniques in Software Engineering (GTTSE'11) (Lecture Notes in Computer Science 7680)*, Ralf Lämmel, João Saraiva, and Joost Visser (Eds.). Springer, Braga, Portugal, 383–430.

[93] Scott A. Vorthmann. 1993. *Modelling and Specifying Name Visibility and Binding Semantics.* Technical Report CMU-CS-93-158. Carnegie Mellon University, Pittsburgh, PA, United States.

[94] Philip Wadler. 1998. The Expression Problem. Java Genericity Mailing List.

[95] Martin Weißbach, Wolf Zimmermann, and Thomas Kühn. 2025. Workflow Net Compositions for the Analysis of Service-Oriented Systems. In *Proceedings of the 11th European Conference on Service-Oriented and Cloud Computing (ESOCC'25) (Lecture Notes in Computer Science 15547)*, Valentina Lenarduzzi and Tomas Cerny (Eds.). Springer, Bolzano, Italy, 235–250.

[96] Claes Wohlin, Martin Höst, and Kennet Henningsson. 2003. Empirical Research Methods in Software Engineering. In *Empirical Methods and Studies in Software Enginering: Experiences from ESERNET*, Reidar Conradi and Alf Inge Wang (Eds.). Springer, 7–23.

[97] Michael Joseph Wolfe. 1982. *Optimizing Supercompilers for Supercomputers.* PhD Thesis. University of Illinois at Urbana-Champaign, Urbana-Champaign, IL, USA.

A Simplified Verification Model: Postorder Semantic Actions

Before facing the complete verification model presented in this paper, we attempted solving the problem with a simplified model. We hereby show the limitations that ultimately lead us opting for a more complex solution. Let us consider the problem of verifying the correctness of attributes within a single semantic role in which the tree visit is performed in postorder: for each node, each of its child nodes is visited first by executing the semantic action associated with the production which generated the child node, then the same operation is repeated on the parent node itself. With this visit strategy, data can only be passed bottom-up in the tree (from child to parent node). Given this premise a semantic action attached to a production $p : X_0 \leftarrow X_1 \cdots X_n$ will read attributes from $X_1 \cdots X_n$, and then write attributes to X_0. Although more complex behaviors are possible, we will consider semantic actions in which there is no reason to:

- read attributes on X_0, as no semantic action which could have written them has been executed yet;
- write attributes to $X_1 \cdots X_n$ as no further semantic action will read them.

Given these restrictions, we can define that $\tau(p[0].a) = t$ if the semantic action attached to p *provides* an attribute a of type t, and $\tau(p[i].a) = t$ with $1 \leq i \leq |p|$ if the semantic action attached to p *requires* an attribute a of a type t' assignable from t ($t' \leq: t$) on X_i.

Now, we can define the concept of *safe attribute access* in the scope of postorder visits. Let us consider a production $p : X_0 \leftarrow X_1 \cdots X_n$ and a semantic action attached to it which accesses the attribute $p[i].a$ (with $1 \leq i \leq n$), and with type requirement $t = \tau(p[i].a)$. We say this access is type safe iff.

$$\forall p' \in P : \; X_i \leftarrow Y_1 \cdots Y_n, \quad \tau(p'[0].a) \leq: t$$

Algorithm 3 Verification of attribute accesses in postorder semantic actions execution

```
E := ∅
for all p ∈ P do
    for i := 1 to |p| do
        for all p′ ∈ P do
            if p′[0] = p[i] then
                for all a ∈ p[i] do
                    if τ(p′[0].a) is not defined then
                        E := E∪ {"p′ does not provide a required by p[i]"}
                    else if not τ(p′[0].a) ≤: τ(p[i].a) then
                        E := E∪ {"type of provided p′[0].a is incompatible with p[i].a"}
                    end if
                end for
            end if
        end for
    end for
end for
if E = ∅ then
    verification succeeded
else
    incorrect attribute accesses have been detected and collected in E
end if
```

or, in other words, if for each production p' with X_i as its left-hand side, an attribute with the same name is provided by the semantic action attached to p' on its left-hand side nonterminal with the same type or a subtype of t.

This lead to a first, albeit limited, algorithm to verify the safety of attribute accesses, that works on semantic actions within a visit executed in postorder: the full pseudocode is shown in Algorithm 3.

A.1 Implementation Aspects and Limitations

An implementation of Algorithm 3 needs a way to gather information about attributes required and provided by the semantic action attached to each production for the role being analyzed. For `Neverlang`, there are two stages of the build process in which it is practical to access this information: during the DSL compilation, while `Neverlang` is performing a translation from the semantic action implementation DSL to the backend code, or after the entire compilation process by inspecting the bytecode through a bytecode analysis tool of choice. Both have been implemented in `Neverlang`. The former adds helper methods to the semantic action classes, returning information about required and provided attributes, in the form of index-name-type triplets; `nlgcheck` queries these methods to implement Algorithm 3.

It is important to note that this text-based approach has limitations due to it lacking any type information which is later generated by the `Java` compiler, and thus it requires additional syntax to properly generate the correct methods—*e.g.* `$0.value: Integer = ...` denotes the semantic action generating an attribute of `Integer` type. However, usage of this syntax extensions when writing semantic actions is optional, and when the developer does not provide any type hints, the attribute type is assumed to be `Object`. As we will discuss later in this section, a more "hands on" approach, inspecting the generated semantic actions bytecode directly, is preferable in general.

This approach represents a solution that was used in the early stages of this work and was successfully tested against simple languages such as the `Java` expression language [21], modified by adding the necessary type annotations. This allows the language to be verified successfully, thus introducing errors in the form of incorrect attribute names and type casts or annotations.

However, such an approach has several limitations that ultimately caused us to deem it inadequate to be used in more concrete scenarios. First, the approach relies on the presence of type hints that the language developers need to manually write in the semantic actions: this requires modification and recompilation of existing modules to make them properly verifiable, and makes the code less readable due to redundant information (the `Java` compiler is aware of the type of each expression without needing type hints). This specific limitation does not regard the algorithm itself but rather its naive original implementation. A much more pressing matter regards the way information about required and provided attributes is extracted from semantic actions: the mere textual presence of an expression accessing an attribute is an imprecise indicator of these events actually happening at runtime. If the statement is, for example, inside a conditional block, its actual execution depends on runtime value of the condition, an aspect that cannot be captured by a simplified textual analysis. Extracting the information regarding required and provided attributes from the bytecode enables the possibility of considering all possible execution paths. More specifically, if different paths provide the same attribute with different types, the resulting type of the attribute can be inferred as the closest common supertype. Similarly, the presence of one branch in which the attribute is not may cause indicate that the attribute is not provided in specific execution scenarios, even though the textual representation indicates otherwise. Finally, if different paths require the same attribute, but with different types, the closest common subtype must be considered. It is arguable that some of these scenarios are unlikely or may suggest improper language implementation, however please recall that this contribution aims at verifying that the composition among **precompiled** language features is feasible, even when they where not originally intended to be used jointly; therefore we believe unlikely scenarios should be taken into consideration.

Last but not least, limiting the solution to postorder visits is not a fair assumption: in any non-trivial language, semantic actions need full control with regards of how the visit must be performed. As a notable example, please consider a `while` statement in an interpreter for a GPL: a strictly postorder (or preorder) visit is not viable, because:

- the node representing the condition must checked (and thus visited) on each iteration;
- the node representing the body may or may not be visited depending on the runtime value of the condition.

B Algorithms

This appendix contains any algorithms that were omitted from the main body of this manuscript for brevity.

B.1 Full CFG Construction Algorithm

Algorithm 4 shows the entire CFG construction algorithm for the complete model not limited to postorder visit.

For the sake of conciseness, this algorithm omits an implementation aspect involving semantic action guards: if multiple semantic actions with different guards are attached to the same production, their respective CFGs are merged in a single CFG representing all the possible branches which can be taken before running Algorithm 4. In case of type guards, edges connecting the entry node of the production CFGs to the entry nodes of the individual semantic actions are tagged with the required attribute types: this allows the visit algorithm to exclude paths which would never be

followed at runtime because of the guards. Please refer to [24] for a full examination of guards and type guards in `Neverlang`.

Algorithm 4 Construction of a role's CFG. $G_i = (V_i, E_i)$ denotes the CFG of the semantic action attached to the production p_i.

create two nodes v_{begin}, v_{end}
$a_{node}(v_{begin}) := a_{node}(v_{end}) := Nop()$
$G_r = (V_r, E_r) := (\{v_{begin}, v_{end}\}, \emptyset)$
$v_{entry}(G_r) := v_{begin}$
$v_{exit}(G_r) := v_{end}$
for all $p_i \in P$ **do**
  $V_r := V_r \cup V_i$
  $E_r := E_r \cup E_i$
  if $p_i[0] = Program$ **then**
    $E_r := E_r \cup \{(v_{begin}, v_{entry}(G_i)), (v_{exit}(G_i), v_{end})\}$
  end if
end for
$E_{entry} := \emptyset$
$E_{exit} := \emptyset$
$L := \emptyset$
for all $p_i \in P$ **do**
  for all $v_{beginEval} \in V_i : \exists k : a_{node}(v_{beginEval}) = BeginEvalMetaAction(k)$ **do**
    find $v_{endEval} \in V_i$ such that $(v_{beginEval}, v_{endEval}) \in E_i$
    $E_r := E_r \setminus \{(v_{beginEval}, v_{endEval})\}$
    for all $p_j \in P : p_j[0] = p_i[k]$ **do**
      generate an unique tag $l \notin L$
      $L := L \cup \{l\}$
      $e_{entry} := (v_{beginEval}, v_{entry}(G_j))$
      $e_{exit} := (v_{exit}(G_j), v_{endEval})$
      $E_{entry} := E_{entry} \cup \{e_{entry}\}$
      $E_{exit} := E_{exit} \cup \{e_{exit}\}$
      $l_{edge}(e_{entry}) := l_{edge}(e_{exit}) := l$
    end for
    $a_{node}(v_{beginEval}) := EnterCtx(k)$
    $a_{node}(v_{endEval}) := LeaveCtx()$
  end for
end for
$E_r := E_r \cup E_{entry} \cup E_{exit}$

B.2 Optimized CFG Visit Algorithm

Algorithm 1 shows the CFG visit algorithm, including the optimizations introduced in §4.5.

C Proofs

This appendix contains any proofs that were omitted from the main body of this manuscript for brevity.

Algorithm 5 Optimized visit of a role's CFG, with cache and jump logic (replaces Algorithm 1)

$\Phi := \{(v_{entry}(G_r), v_{entry}(G_r), \epsilon, \langle\emptyset\rangle, \epsilon)\}$ ▷ Set of states to visit
$\Gamma := \emptyset$ ▷ Set of visited states
$U := \emptyset$ ▷ Found violations
$f_c := \emptyset$ ▷ Cache for jumps
while $\Phi \neq \emptyset$ **do**
  pick any $(v, w, S, C, S_c) \in \Phi$ and remove it from Φ
  if $(v, C, S) \notin \Gamma$ **then** ▷ Path condition (3)
    $\Gamma := \Gamma \cup (v, C, S)$
    $(C', U') := f_{action}(C, a_{node}(v))$
    $U := U \cup U'$
    for all $(v, v') \in E_r$ **do**
      $S' := f_{edge}(S, (v, v'))$
      if $\exists S'$ **and** $\nexists l_1, l_2, l_3 \in S : l_1 = l_2 = l_3$ **then** ▷ Path validity and (2)
        if $(v, v') \in E_{entry}$ **then** ▷ Then v' is a nonterminal entry
          if $\exists f_c(v', head(C))$ **then**
            $(v_{out}, C_{outs}, C_{taken}) := f_c(v', head(C))$
            for all $C_{out} \in C_{outs}$ **do**
              $\Phi := \Phi \cup \{(v_{out}, wv', S', merge_h(C, C_{out}), push(S_c, head(C)))\}$
            end for
            $f_c(v', head(C)) := (v_{out}, C_{outs}, C_{taken} \cup C)$
          else
            $\Phi := \Phi \cup \{(v', wv', S', C', push(S_c, head(C)))\}$
          end if
        else if $(v, v') \in E_{exit}$ **then** ▷ Then v is a nonterminal exit
          find v_{in} such that $\exists v_x : l_{edge}((v_x, v_{in})) = l_{edge}(v, v')$
          if $\exists f_c(v_{in}, top(S_c))$ **then**
            $(v_{out}, C_{outs}, C_{taken}) := f_c(v', head(C))$
            if $head(C) \notin C_{outs}$ **then**
              for all $C_t \in C_{taken}$ **do**
                $\Phi := \Phi \cup \{(v', wv', S', merge_h(C_t, head(C)))\}$
              end for
            end if
            $f_c(v', head(C)) := (v_{out}, C_{outs} \cup \{head(C)\}, C_{taken})$
          else
            $f_c(v_{in}, top(S_c)) := (v, \{head(C)\}, \emptyset)$
          end if
          $\Phi := \Phi \cup \{(v', wv', S', C', pop(S_c))\}$
        else
          $\Phi := \Phi \cup \{(v', wv', S', C', S_c)\}$
        end if
      end if
    end for
  end if
end while

Lemma 1. *f_{pCtx} can be evaluated "piecewise" on a path by composing evaluations of f_{pCtx} on parts of the path:*

$$\forall w_1, w_2 \in V_r^*, C \in \mathcal{C}, \quad f_{pCtx}(w_1 w_2, C) = f_{pCtx}(w_2, f_{pCtx}(w_1, C))$$

Proof. This can be proven by induction on w_1. The base case $w_1 = \epsilon$ is trivial, as by definition $f_{pCtx}(\epsilon, C) = C$ and the goal is reduced to an identity. For the inductive step, given w_1, we assume

$$\forall w_2 \in V_r^*, C' \in \mathcal{C}, \quad f_{pCtx}(w_1 w_2, C') = f_{pCtx}(w_2, f_{pCtx}(w_1, C')) \tag{6}$$

and we must prove that

$$\forall v \in V_r, w_2 \in V_r^*, C \in \mathcal{C}, \quad f_{pCtx}(v w_1 w_2, C) = f_{pCtx}(w_2, f_{pCtx}(v w_1, C))$$

By definition,

$$f_{pCtx}(v w_1 w_2, C) = f_{pCtx}(w_1 w_2, f_{aCtx}(C, a_{node}(v))) =$$

applying the inductive hypothesis (6) with $C' = f_{aCtx}(C, a_{node}(v))$,

$$= f_{pCtx}(w_2, f_{pCtx}(w_1, f_{aCtx}(C, a_{node}(v)))) =$$

and finally, applying once more the definition of f_{pCtx}, this time in "reverse" for the $v w_1$ path,

$$= f_{pCtx}(w_2, f_{pCtx}(v w_1, C))$$

thus proving the property. □

Lemma 6. *Given an integer $k > 0$ and two production context stacks sharing the same top k items, the evaluation context stacks obtained by evaluating a balanced path prefix which does not contain CopyAttr nor PropagateAttrs actions share the top k items:*

$$\forall w \in W_{pb}, \forall C_1, C_2 \in \mathcal{C}, k \in \mathbb{N}$$
$$top_k(C_1) = top_k(C_2) \implies top_k(f_{pCtx}(w, C_1)) = top_k(f_{pCtx}(w, C_2))$$

Proof. Due to the hypothesis $w \in W_{pb}$, the action sequence associated with w is either one of the following cases.

- If $w = \epsilon$, the implication is trivial by definition of f_{pCtx}, as $\forall C,\ f_{pCtx}(\epsilon, C) = C$.
- If $w = v w'$, with $w' \in W_{pb}$ and $a_{node}(v) = a$ with $a \in A \setminus \{LeaveCtx()\}$ then the implication is proved by induction, assuming

  $$\forall C_1, C_2, k, \quad top_k(C_1) = top_k(C_2) \implies top_k(f_{pCtx}(w', C_1)) = top_k(f_{pCtx}(w', C_2)) \tag{7}$$

  for a fixed w' and proving that

  $$\forall C_1, C_2, k, \quad top_k(C_1) = top_k(C_2) \implies top_k(f_{pCtx}(v w', C_1)) = top_k(f_{pCtx}(v w', C_2))$$

  Assuming $top(C_1) = top(C_2)$ and by definition of f_{pCtx}:

  $$top_k(f_{pCtx}(w', f_{aCtx}(C_1, a))) = top_k(f_{pCtx}(w', f_{aCtx}(C_2, a))) \tag{8}$$

  For $a \in \{ReadAttr, CheckAttrType, Nop\}$, it is true that $f_{aCtx}(C', a) = C'\ \forall C'$, thus by substituting $f_{aCtx}(C_1, a) = C_1$ and $f_{aCtx}(C_2, a) = C_2$ we obtain the inductive hypothesis (7).
  Otherwise either $a = WriteAttr(i, \eta, t)$ or $a = EnterCtx(i)$.
  In the former case, applying the definition of f_{action} for $WriteAttr$, (8) becomes

  $$top_k(f_{pCtx}(w', put(C_1, i, \eta, t))) = top_k(f_{pCtx}(w', put(C_2, i, \eta, t)))$$

  Then, applying the definition of put on $C_1 = \langle c_1, \ldots, c_n \rangle$ and $C_2 = \langle c'_1, \ldots, c'_n \rangle$:

  $$top_k(f_{pCtx}(w', \langle c_1, \ldots, c_{put1}(c_{n-1}, k_n, i, \eta, t), c_{put2}(c_n, i, \eta, t) \rangle)) =$$
  $$top_k(f_{pCtx}(w', \langle c'_1, \ldots, c_{put1}(c'_{n-1}, k'_n, i, \eta, t), c_{put2}(c'_n, i, \eta, t) \rangle))$$

Since $top_k(C_1) = top_k(C_2), \forall i \in \mathbb{N} : \; k \leq i \leq n, \quad c_i = c'_i$, therefore, the arguments of the $f_{pCtx}(w', \ldots)$ in the two members of (8) share the same top_k, proving the inductive step. Finally, when $a = EnterCtx(i)$, (8) can be rewritten as

$$top_k(f_{pCtx}(w', push(C_1, (i, \emptyset)))) = top_k(f_{pCtx}(w', push(C_2, (i, \emptyset))))$$

By the definitions of $push$ and top_k, and the hypothesis $top_k(C_1) = top_k(C_2)$, it follows that

$$top_{k+1}(push(C_1, (i, \emptyset))) = top_{k+1}(push(C_2, (i, \emptyset)))$$

and, by applying the inductive hypothesis (7) to $push(C_1, (i, \emptyset))$ and $push(C_2, (i, \emptyset))$:

$$top_{k+1}(f_{pCtx}(w', push(C_1, (i, \emptyset)))) = top_{k+1}(f_{pCtx}(w', push(C_2, (i, \emptyset))))$$

The equality of top_{k+1} trivially implies the equality of top_k, thus proving (8).

- If $w = w'v$, the proof is similar. The same inductive hypothesis holds. According to the definition of f_{pCtx} and Lemma 1 the goal $top_k(C_1) = top_k(C_2)$ can be rewritten as:

$$top_k(f_{pCtx}(w'v, C_1)) = top_k(f_{pCtx}(w'v, C_2))$$
$$top_k(f_{aCtx}(f_{pCtx}(w', C_1), a)) = top_k(f_{aCtx}(f_{pCtx}(w', C_2), a))$$

  Let $C_3 = f_{pCtx}(w', C_1)$ and $C_4 = f_{pCtx}(w', C_2)$; from the inductive hypothesis (7) follows that $top_k(C_3) = top_k(C_4)$. As seen in the previous case, only $WriteAttr$ and $EnterCtx$ can modify the context stack and both actions preserve the equality of the top_k, thus from the definition of f_{action} follows that

$$top_k(f_{aCtx}(C_3, a)) = top_k(f_{aCtx}(C_4, a))$$

- If $w = v_1 w' v_2$ with $w' \in W_{pb}$, $a_{node}(v_1) = a_1 = EnterCtx(i)$, and $a_{node}(v_2) = a_2 = LeaveCtx()$. The inductive hypothesis (7) still holds and the goal is to prove that

$$top_k(f_{pCtx}(v_1 w' v_2, C_1)) = top_k(f_{pCtx}(v_1 w' v_2, C_2))$$

  for any two $C_1 = \langle c_0, \ldots, c_n \rangle$ and $C_2 = \langle c'_0, \ldots, c'_n \rangle$ such that $top_k(C_1) = top_k(C_2)$. This goal can be rewritten as:

$$top_k(f_{aCtx}(f_{pCtx}(w', f_{aCtx}(C_1, a_1)), a_2)) = top_k(f_{aCtx}(f_{pCtx}(w', f_{aCtx}(C_2, a_1)), a_2))$$
$$top_k(pop(f_{pCtx}(w', push(C_1, (i, \emptyset))))) = top_k(pop(f_{pCtx}(w', push(C_2, (i, \emptyset)))))$$
$$top_k(pop(f_{pCtx}(w', \langle c_0, \ldots, c_n, (i, \emptyset) \rangle))) = top_k(pop(f_{pCtx}(w', \langle c'_0, \ldots, c'_n, (i, \emptyset) \rangle)))$$

  Since $top_k(C_1) = top_k(C_2)$, $c_i = c'_i \; \forall i \in \{k, \ldots, n\}$, and thus

$$top_{k+1}(\langle c_0, \ldots, c_n, (i, \emptyset) \rangle) = top_{k+1}(\langle c'_0, \ldots, c'_n, (i, \emptyset) \rangle)$$

  This satisfies the premise of the inductive hypothesis (7) (notice that it holds for any k, and $k + 1$ in particular), therefore

$$top_{k+1}(f_{pCtx}(\langle c_0, \ldots, c_n, (i, \emptyset) \rangle)) = top_{k+1}(f_{pCtx}(\langle c'_0, \ldots, c'_n, (i, \emptyset) \rangle))$$

  Applying pop to each of the two stacks removes the topmost item preserving the rest; therefore even though the equality no longer holds for top_{k+1}, it still holds for top_k, thus proving the goal.

□

Lemma 8. *Given any two production context stacks, evaluating a balanced path prefix which does not contain CopyAttr or PropagateAttrs actions, wrapped between two EnterCtx nodes produces context stacks with the same head.*

$$\forall C_1, C_2 \in \mathcal{C}, w \in W_{pb}, \quad v_{sep} \in V_r^* : \; \exists i : a_{node}(v_{sep}) = EnterCtx(i),$$

$$head(f_{pCtx}(v_{sep}wv_{sep}, C_1)) = head(f_{pCtx}(v_{sep}wv_{sep}, C_2)) \tag{9}$$

Proof. The lemma is proven by proving each of the following cases.

- If $w = \epsilon$, then the goal (9) becomes

  $$head(f_{pCtx}(v_{sep}v_{sep}, C_1)) = head(f_{pCtx}(v_{sep}v_{sep}, C_2))$$

  By definition of f_{pCtx}:

  $$head(f_{aCtx}(f_{aCtx}(C_1, a_s), a_s)) = head(f_{aCtx}(f_{aCtx}(C_2, a_s), a_s)) \quad \text{with } a_s = EnterCtx(i)$$

  By definition of f_{aCtx} and f_{action} for $EnterCtx$,[15]

  $$head(push(push(C_1, (i, \emptyset)), (i, \emptyset))) = head(push(push(C_2, (i, \emptyset)), (i, \emptyset)))$$

  Let $C_1 = \langle c_0, \ldots, c_n \rangle$ and $C_1 = \langle c'_0, \ldots, c'_n \rangle$; by definition of $push$,

  $$head(\langle c_0, \ldots c_n, (i, \emptyset), (i, \emptyset) \rangle) = head(\langle c'_0, \ldots c'_n, (i, \emptyset), (i, \emptyset) \rangle) =$$

  which is proven by Lemma 2.
- If $w = vw'$, with $w' \in W_{pb}$ and $a_{node}(v) = a$ with $a \in A \setminus \{LeaveCtx()\}$ then we must prove that, $\forall C_1, C_2 \in C$,

  $$head(f_{pCtx}(v_{sep}vw'v_{sep}, C_1)) = head(f_{pCtx}(v_{sep}vw'v_{sep}, C_2)) \tag{10}$$

  Let's expand both sides of the equality using a generic C instead of C_1 and C_2 by applying Lemma 1:

  $$head(f_{pCtx}(v_{sep}vw'v_{sep}, C)) =$$
  $$head(f_{pCtx}(vw'v_{sep}, f_{pCtx}(v_{sep}, C))) =$$
  $$head(f_{pCtx}(v_{sep}, f_{pCtx}(vw', f_{pCtx}(v_{sep}, C)))) =$$

  by definition of f_{pCtx} and then of f_{aCtx} for $EnterCtx(i)$,

  $$= head(f_{aCtx}(f_{pCtx}(vw', f_{aCtx}(C, a_s)), a_s)) =$$
  $$head(push(f_{pCtx}(vw', push(C, (i, \emptyset))), (i, \emptyset)))$$

  The goal (10) can therefore be rewritten as

  $$head(push(f_{pCtx}(vw', push(C_1, (i, \emptyset))), (i, \emptyset))) =$$
  $$head(push(f_{pCtx}(vw', push(C_2, (i, \emptyset))), (i, \emptyset)))$$

  This can be proven through Lemma 2, showing that the last two items of the context stacks passed to *head* in the two members of the equality are equal. By definition of *push*, the last item is $(i, \emptyset)$ at both members; also by definition of *push* we know the second-last item is the *top* of the argument of *push*, therefore we must prove that

  $$top(f_{pCtx}(vw', push(C_1, (i, \emptyset)))) = top(f_{pCtx}(vw', push(C_2, (i, \emptyset))))$$

  By definition, vw' is a balanced path prefix because it is formed by the concatenation of a balanced path prefix w' and a node v such that $a_{node}(v) \neq LeaveCtx()$. Moreover, $top(push(C_1, (i, \emptyset))) = top(push(C_2, (i, \emptyset)))$ by definition of *top* and *push*. Therefore, the goal is proven by applying Lemma 6 with $k = 1$, path vw' and context stacks $push(C_1, (i, \emptyset))$ and $push(C_2, (i, \emptyset))$.
- If $w = w'v$, the proof is identical to the previous case, as the considerations made on vw' can be made on $w'v$ as well.

[15] Notice that for brevity, we will keep using the equivalence $a_s = EnterCtx(i)$ throughout this proof.

- If $w = v_1 w' v_2$ with $w' \in W_{pb}$, $a_{node}(v_1) = a_1 = EnterCtx(j)$, and $a_{node}(v_2) = a_2 = LeaveCtx()$:

$$head(f_{pCtx}(v_{sep} v_1 w' v_2 v_{sep}, C_1)) = head(f_{pCtx}(v_{sep} v_1 w' v_2 v_{sep}, C_2)) \tag{11}$$

Let us expand both members of the equality, using C in place of C_1 and C_2 respectively:

$$head(f_{pCtx}(v_{sep} v_1 w' v_2 v_{sep}, C)) =$$
$$head(f_{aCtx}(f_{aCtx}(f_{pCtx}(w', f_{aCtx}(f_{aCtx}(C, a_s), a_1)), a_2), a_s)) =$$

recalling that $a_s = EnterCtx(i)$ and thus $f_{aCtx}(C', a_s) = push(C', (i, \emptyset))$,

$$= head(push(f_{aCtx}(f_{pCtx}(w', f_{aCtx}(push(C, (i, \emptyset)), a_1)), a_2), (i, \emptyset))) =$$

recalling $a_1 = EnterCtx(j)$ and $a_2 = LeaveCtx()$, thus $f_{aCtx}(C', a_1) = push(C', (j, \emptyset))$ and $f_{aCtx}(C', a_2) = pop(C')$,

$$= head(push(pop(f_{pCtx}(w', push(push(C, (i, \emptyset)), (j, \emptyset)))), (i, \emptyset)))$$

With $C_1 = \langle c_0, \ldots, c_n \rangle$ and $C_2 = \langle c'_0, \ldots, c'_n \rangle$, the goal (11) can be rewritten as

$$head(push(pop(f_{pCtx}(w', \langle c_0, \ldots, c_n, (i, \emptyset), (j, \emptyset) \rangle)), (i, \emptyset))) =$$
$$head(push(pop(f_{pCtx}(w', \langle c'_0, \ldots, c'_n, (i, \emptyset), (j, \emptyset) \rangle)), (i, \emptyset)))$$

Let $C_3 = \langle c_0, \ldots, c_n, (i, \emptyset), (j, \emptyset) \rangle$ and $C_4 = \langle c'_0, \ldots, c'_n, (i, \emptyset), (j, \emptyset) \rangle$. Then, by definition of top_k, $top_2(C_3) = top_2(C_4)$. Applying Lemma 6:

$$top_2(f_{pCtx}(w', C_3)) = top_2(f_{pCtx}(w', C_4))$$

Thus, we must prove that

$$head(push(pop(f_{pCtx}(w', C_3)), (i, \emptyset))) = head(push(pop(f_{pCtx}(w', C_4)), (i, \emptyset)))$$

Applying pop to $f_{pCtx}(w', C_3)$ and to $f_{pCtx}(w', C_4)$ produces, respectively, two stacks C_5 and C_6 such that $top(C_5) = top(C_6)$, reducing the goal to

$$head(push(C_5, (i, \emptyset))) = head(push(C_6, (i, \emptyset)))$$

which is trivially proven by definition of $push$ and $head$ and by the equality $top(C_5) = top(C_6)$. □